编译原理
13. 垃圾回收

rainoftime.github.io
浙江大学
计算机科学与技术学院

Content

Outline

- Mark-and-sweep collection
- Reference counting
- Copying collection
- Generational Collection (书上有，但不考)
- Incremental Collection (书上有，但不考)
- Interface to the Compiler

Outline

Introduction

Mark-and-Sweep

Reference Count

Copying Collection

Interface to the Compiler

Memory Management

- **Problems with manual management**
  - Double frees, use-after frees, …
  - Memory leak, fragmentation, …
- **Storage bugs are hard to find**
  - A bug can lead to a visible effect far away in time and program text from the source
- **How to manage the memory**?
  - For performance, productivity, safety & security

Major Areas of Memory

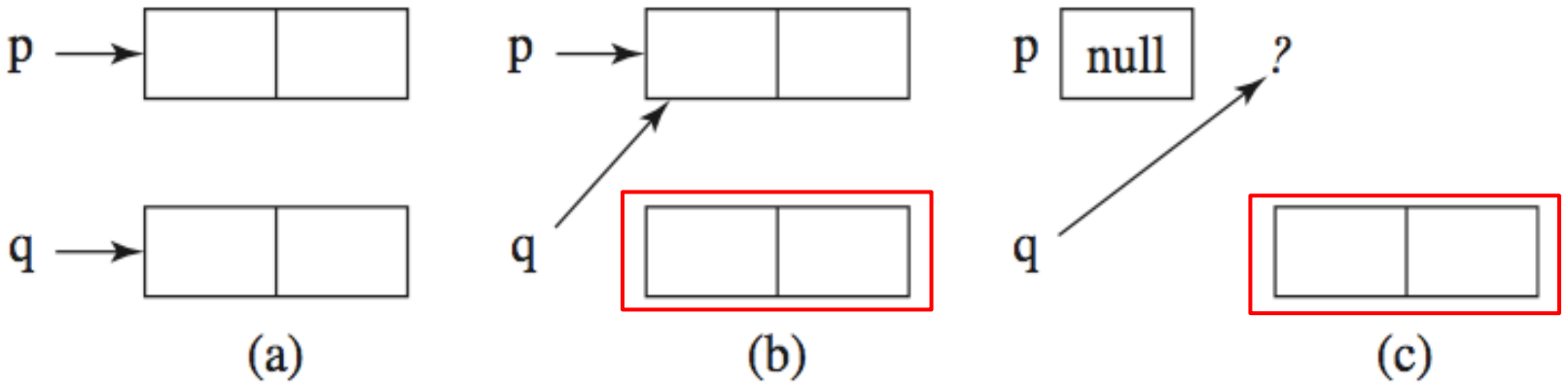
- **The static area:** Allocated at compile time
- **The run-time stack**
  - Activation records
  - Used for managing function calls and returns
- **The heap:** Dynamically allocated objects

Garbage Collection: What

- Garbage: Allocated but no longer used heap storage

```
class node {
    int value;
    node next;
}
node p, q;
```

```
p = new node();
q = new node();
q = p;
delete p;
```



Garbage Collection: What

- Garbage: Allocated but no longer used heap storage

- **Garbage collection:** automatically frees storage which is not used by the program any more
  - Is part of the runtime system
  - First application in LISP (McCarthy 1960)

- A garbage collector has two phases
  - Garbage detection: finds which objects are alive and which dead
  - Garbage reclamation: deallocates dead objects

Garbage Collection: What

- **What is an ideal garbage collector?**
  - **Safe**: only garbage is reclaimed
  - "**Complete**": almost all garbage is collected
  - **Low overhead** in time and sped
  - Short **pause time** (the program waists for the collector)
  - **Parallel**: able to utilize additional cores
  - **Simple** 😄
  - **Easy to use collected free space**
  - **…?**

These are difficult and often conflicting requirements!

Garbage Collection: How

- Garbage: Allocated but no longer used heap storage

- **Question**: When is memory cell *M* not any longer used?
  - Let *P* be any program not using *M*
  - New program sketch:

    Execute *P*; Use *M*;
  - Hence: *M* used $\Leftrightarrow$ *P* terminates
  - We are doomed: halting problem!

- So "last use / non longer used" is undecidable!

Garbage Collection: How

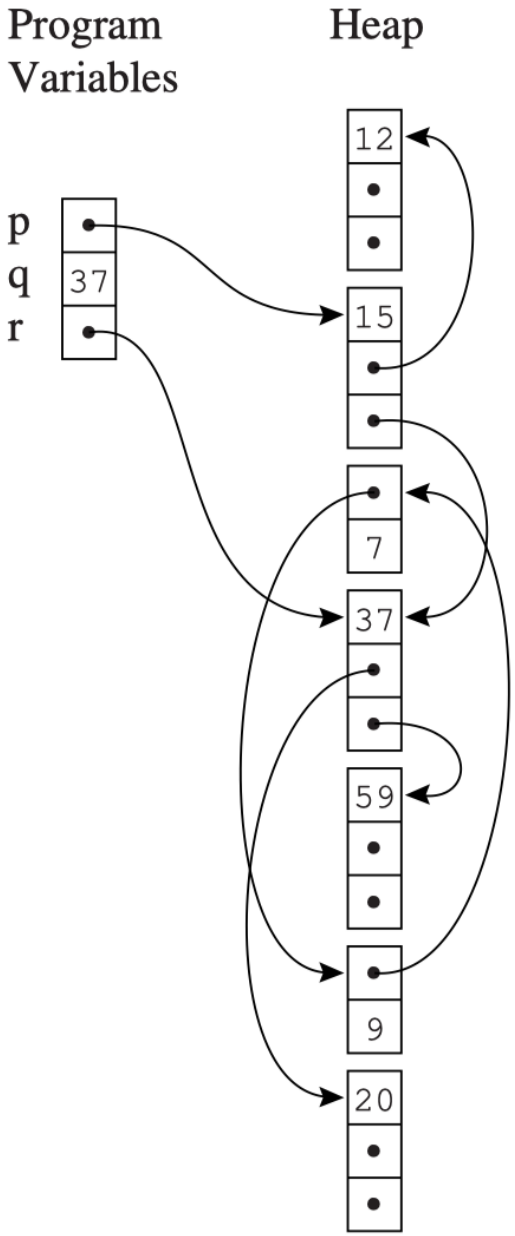
- Garbage: Allocated but no longer used heap storage
- It is undecidable whether an object is garbage
  - Need to rely on a conservative approximation
  - Such that the GC is SAFE (only garbage is reclaimed)
- **Idea**: Use reachability information as "approximation"
  - Heap-allocated records unreachable by any chain of pointers from program variables are garbage
  - By "conservative approximation", we mean

Unreachable ➔ not live (no longer used)

Overview of GC Techniques

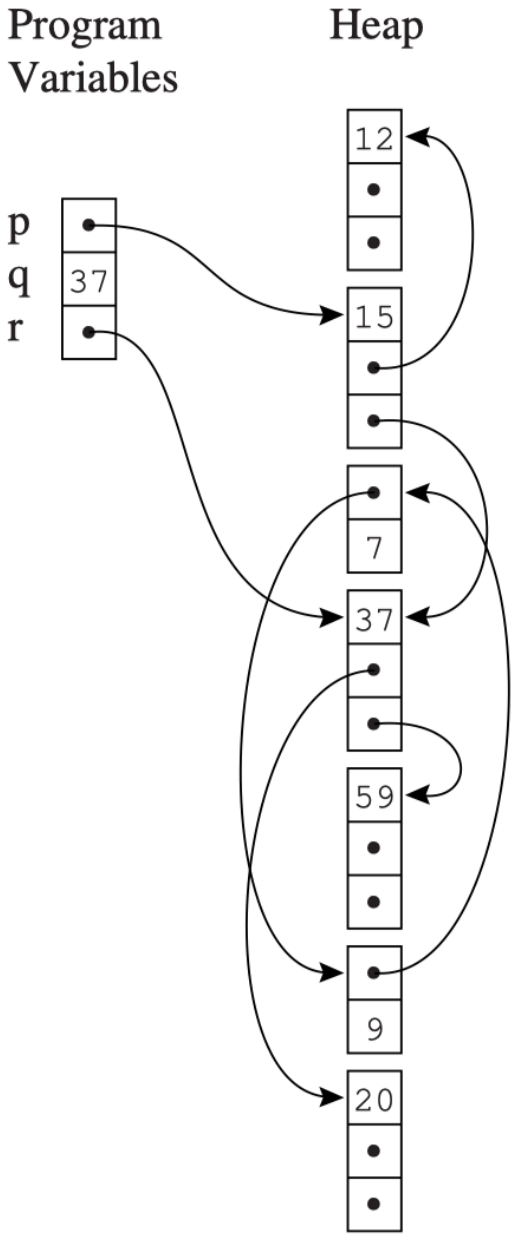
- **Reference counting**
  - Directly keeps track of live cells
  - GC takes place whenever heap block is allocated
  - Doesn't detect all garbage
- **Tracing**
  - GC takes place and identifies live cells when a request for memory fails
  - Mark-sweep
  - Copy collection
- **Modern techniques**: generational GC, etc.

Basic Data Structure: Directed Graph



- **Directed graph**
  - nodes: program variables and heap-allocated records
  - edges: pointers
- **Root**: the program variables are roots of this graph.
  - Registers
  - Local vars/formal parameters on stack
  - global variables
- A node is **reachable** if there is a path of directed edges r -> ... -> n
  - r: some root

Basic Data Structure: Directed Graph



- **Directed graph**
  - nodes: program variables and heap-allocated records
  - edges: pointers

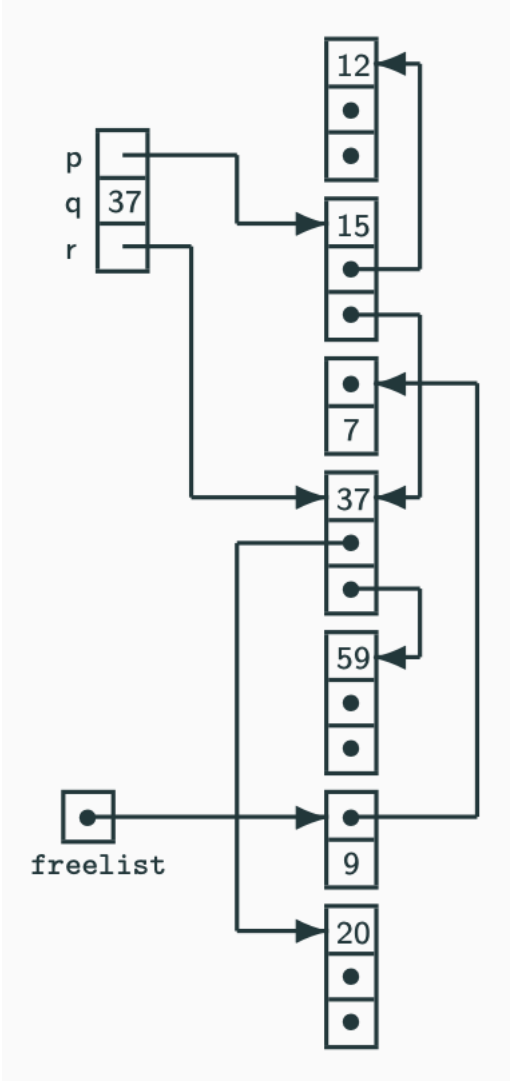
- On understanding the "pointers"
  - p points to a record y: we mean the value of p is the address of y (let y be the name /identifier of the record)

We may use "p" to refer to either the pointer **OR** the record it points to (as in Tiger book…)

Additional Basic Data Structures: Freelist

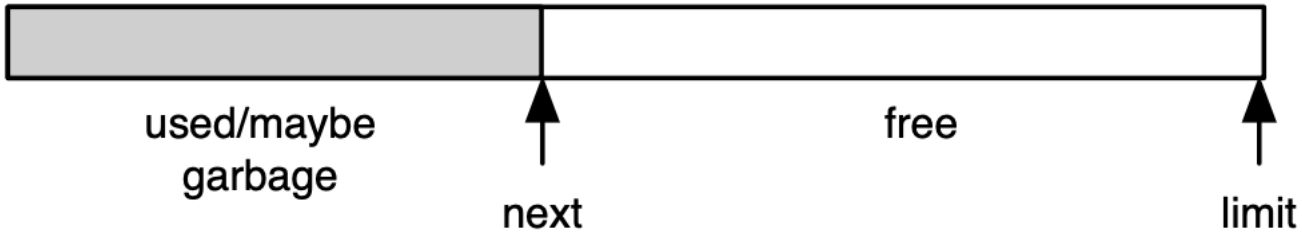
- The memory manager must know which parts of the heap are free and allocated

- The Freelist
  - Free blocks are stored in a free list (a linked list of heap blocks)
  - Used by several GC algorithms (e.g., marked-and-sweep)

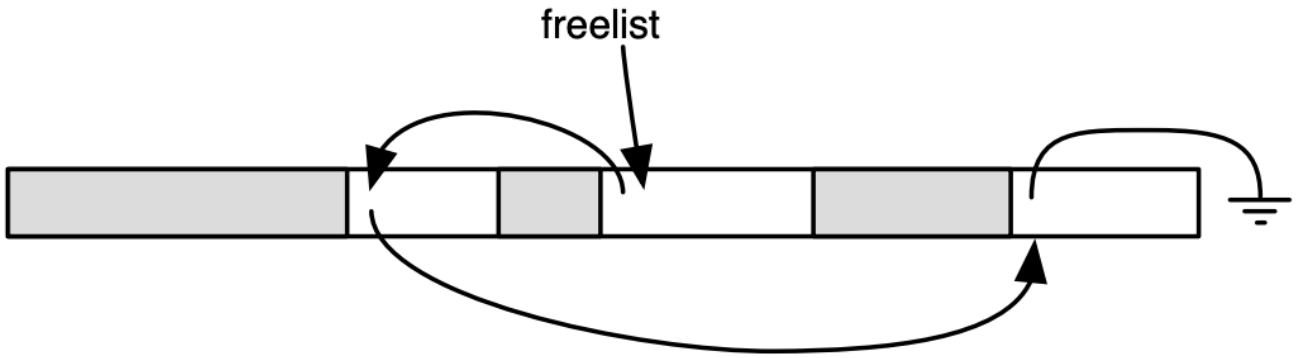


Background: Different Ways for Allocation

- The memory manager must know which parts of the heap are free and allocated
  - **Linear allocation**



  - **Freelist allocation**



1. Mark-and-Sweep

- **Mark-and-Sweep**
- **Explicit Stack**
- **Pointer Reversal**
- **Memory Fragmentation**

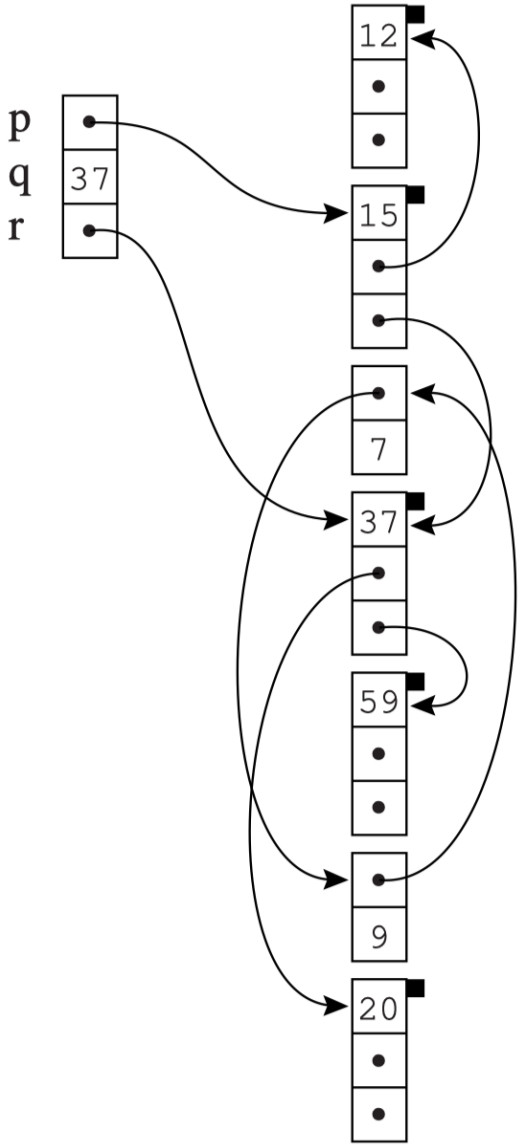
Mark-and-Sweep

- **Mark**
  - Search the graph from the roots (program variables)
  - Mark all the nodes searched

- **Sweep**
  - Sweep the entire heap by a linear scan, putting the unmarked nodes to a freelist
  - Unmark marked nodes

- GC is triggered by a lack of memory, and must complete before the program can be resumed ("**stop the world**").

Mark-and-Sweep: Mark



(a) Marked

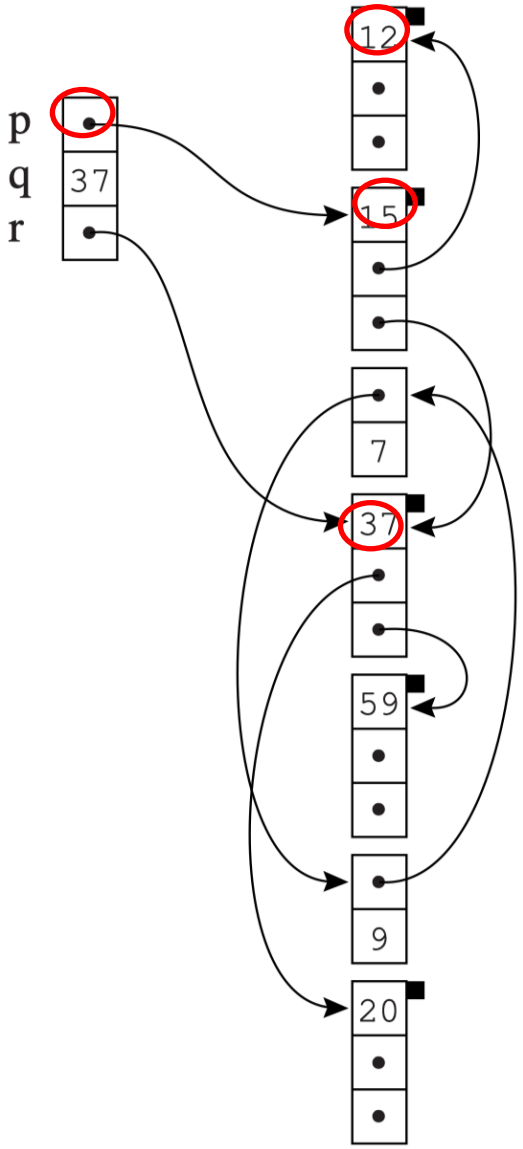
- A depth-first search can mark all the reachable nodes.

```
for each root v
   DFS(v)

function DFS(x)
if x is a pointer into the heap
   if record x is not marked
      mark x
      for each field fi of record x
            DFS(x.fi)
```

??x到底是pointe
还是record

- x的值是某record的地址，换句话说它指向该record
- 如果给该record一个名字(比如y), 就相当于说”x is a pointer and it points to record y” !

Mark-and-Sweep: Mark

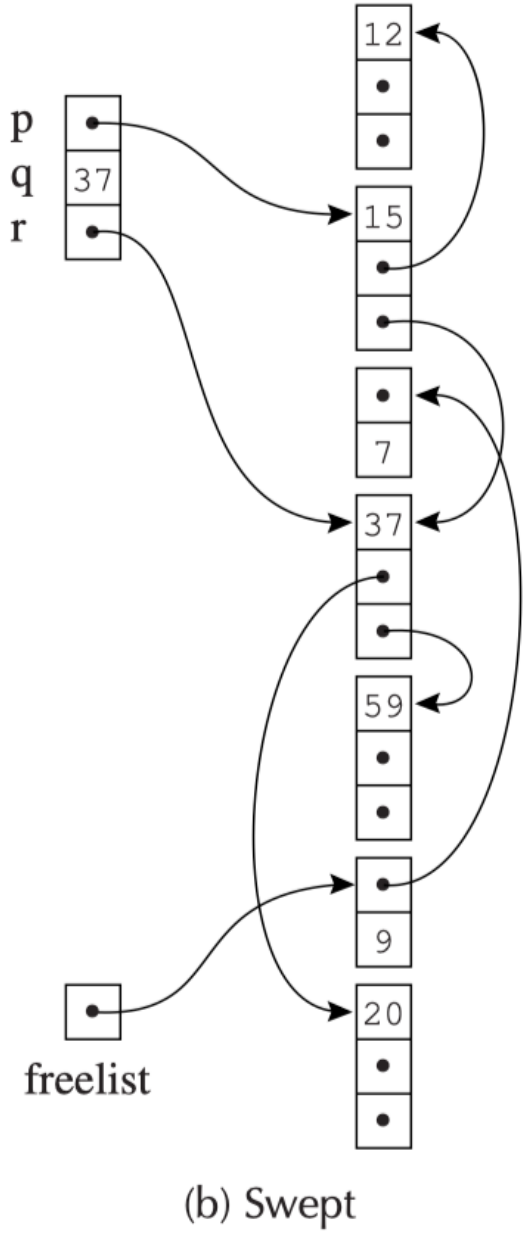


(a) Marked

- A depth-first search can mark all the reachable nodes

```
for each root v
   DFS(v)

function DFS(x)
if x is a pointer into the heap
   if record x is not marked
      mark x
      for each field fi of record x
            DFS(x.fi)
```

Mark-and-Sweep: Sweep

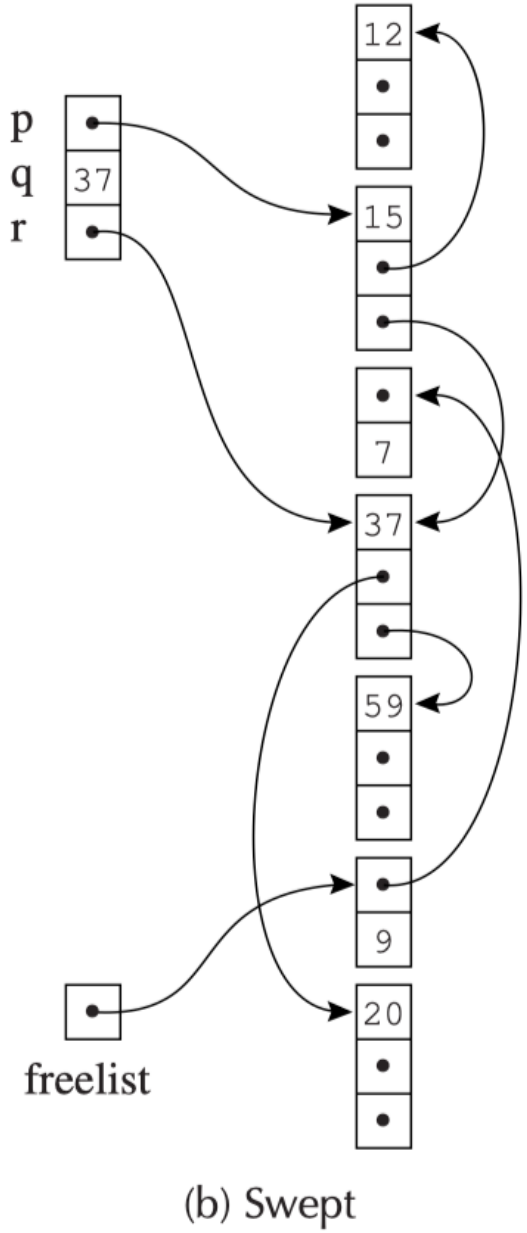


(b) Swept

- **Sweep** the entire heap, from its first address to its last
  - Unmark all the marked nodes for the next garbage collection.

```
p ← first address in heap
while p < last address in heap
    if record p is marked
        unmark p
    else let f1 be the first field in p
        p.f1 ← freelist
        freelist ← p
    p ← p + (size of record p)
```

Mark-and-Sweep: Sweep



(b) Swept

- **Sweep** the entire heap, from its first address to its last
  - Find unmarked nods (garbage)
  - Link them together in *freelist*.

```
p ← first address in heap
while p < last address in heap
    if record p is marked
        unmark p
    else let f1 be the first field in p
        p.f1 ← freelist
        freelist ← p
    p ← p + (size of record p)
```

E.g., 7 and 9 are added to the freelist

Mark-and-Sweep Collection

Mark phase:

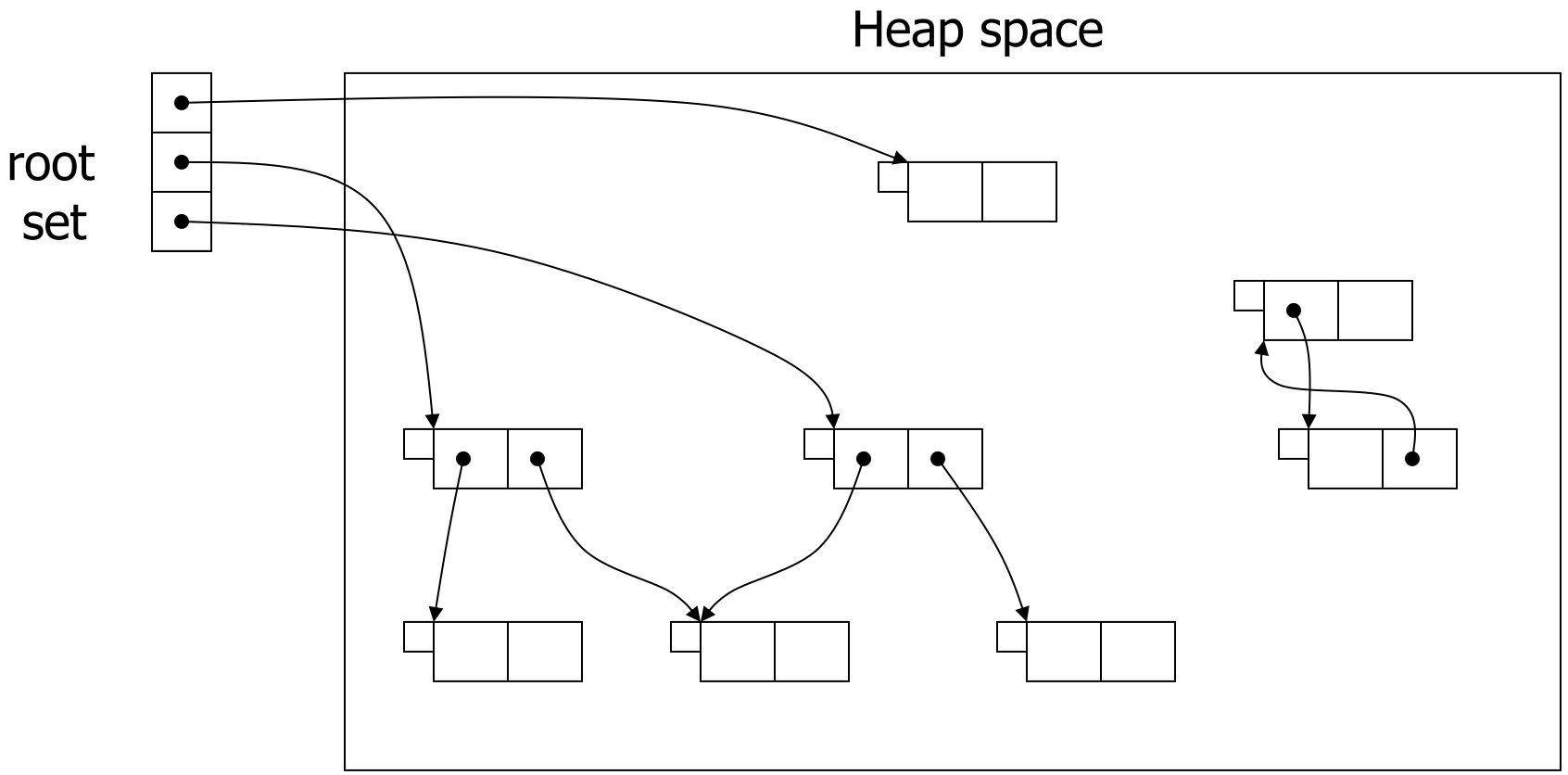
```
for each root v
    DFS(v)

function DFS(x)
   if x is a pointer into the heap
    if record x is not marked
        mark x
        for each field fi of record x
            DFS(x.fi)
```

Sweep phase:

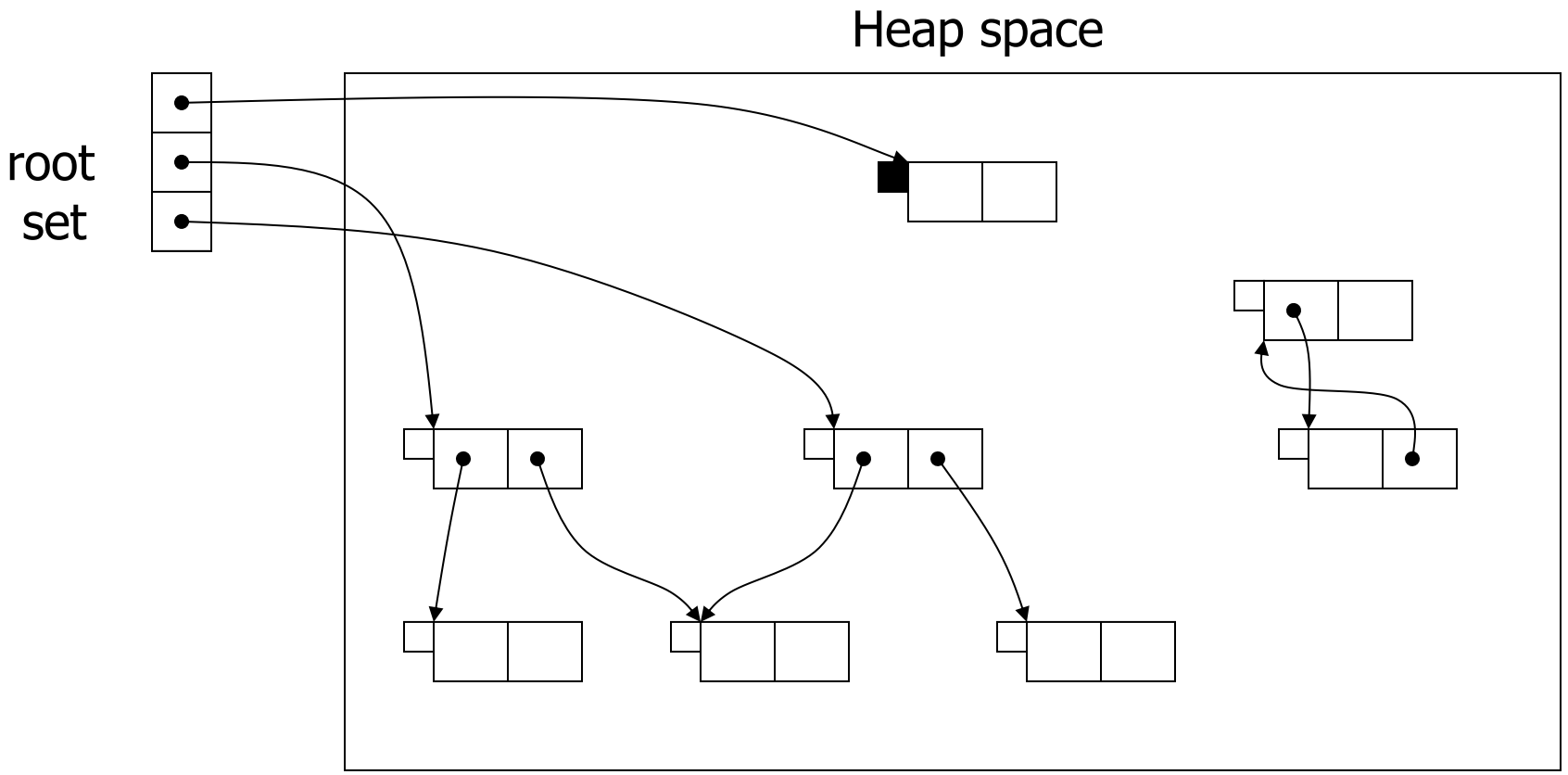
```
p ← first address in heap
 while p < last address in heap
    if record p is marked
            unmark p
    else let f1 be the first field in p
            p.f1 ← freelist
            freelist ← p
    p ← p + (size of record p)
```

- After GC, the program resumes execution.
- Whenever it wants to heap-allocate a new record, it gets a record from the freelist
- Do garbage collection again when freelist is empty !

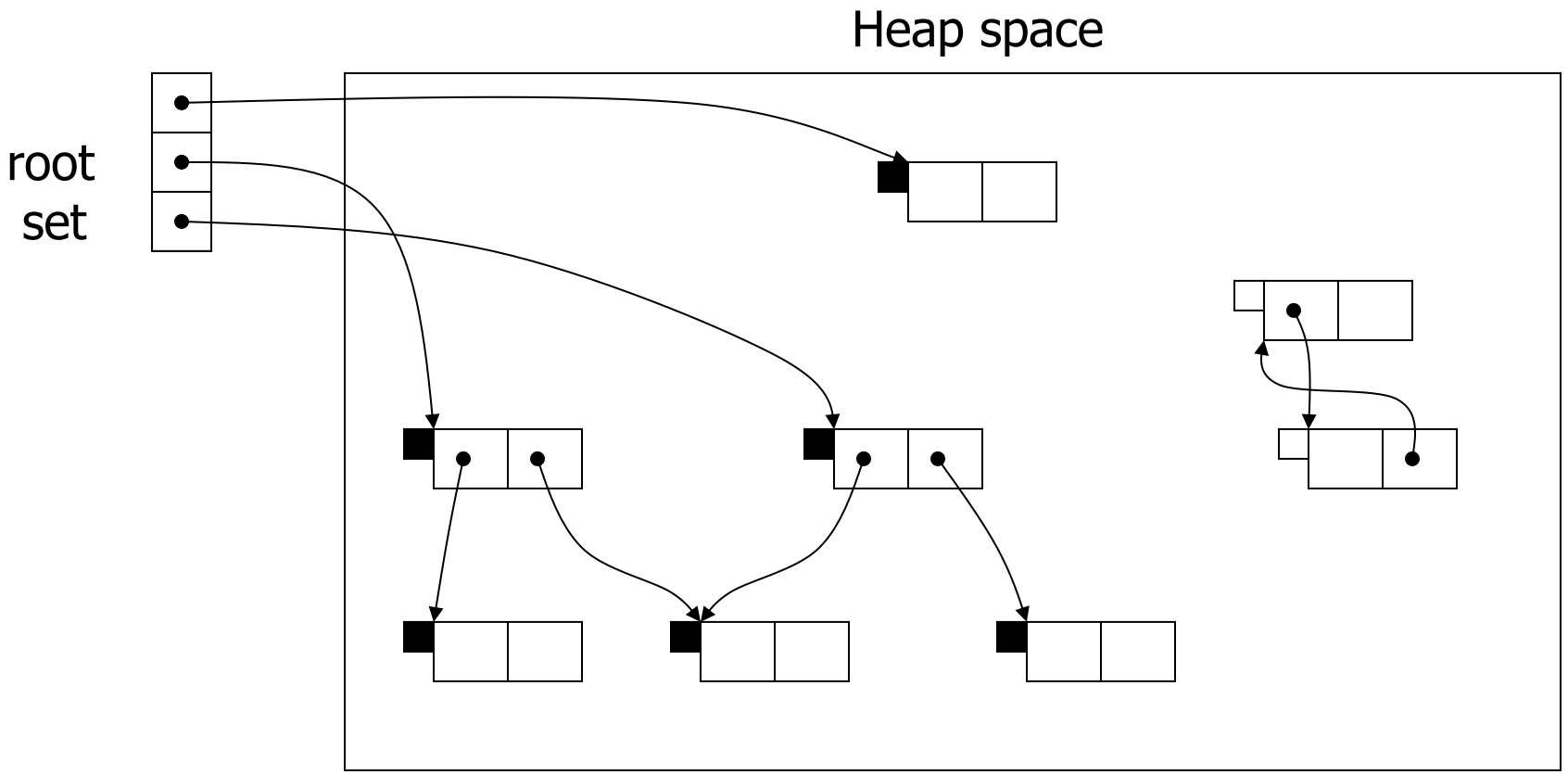
Example: Mark-and-Sweep



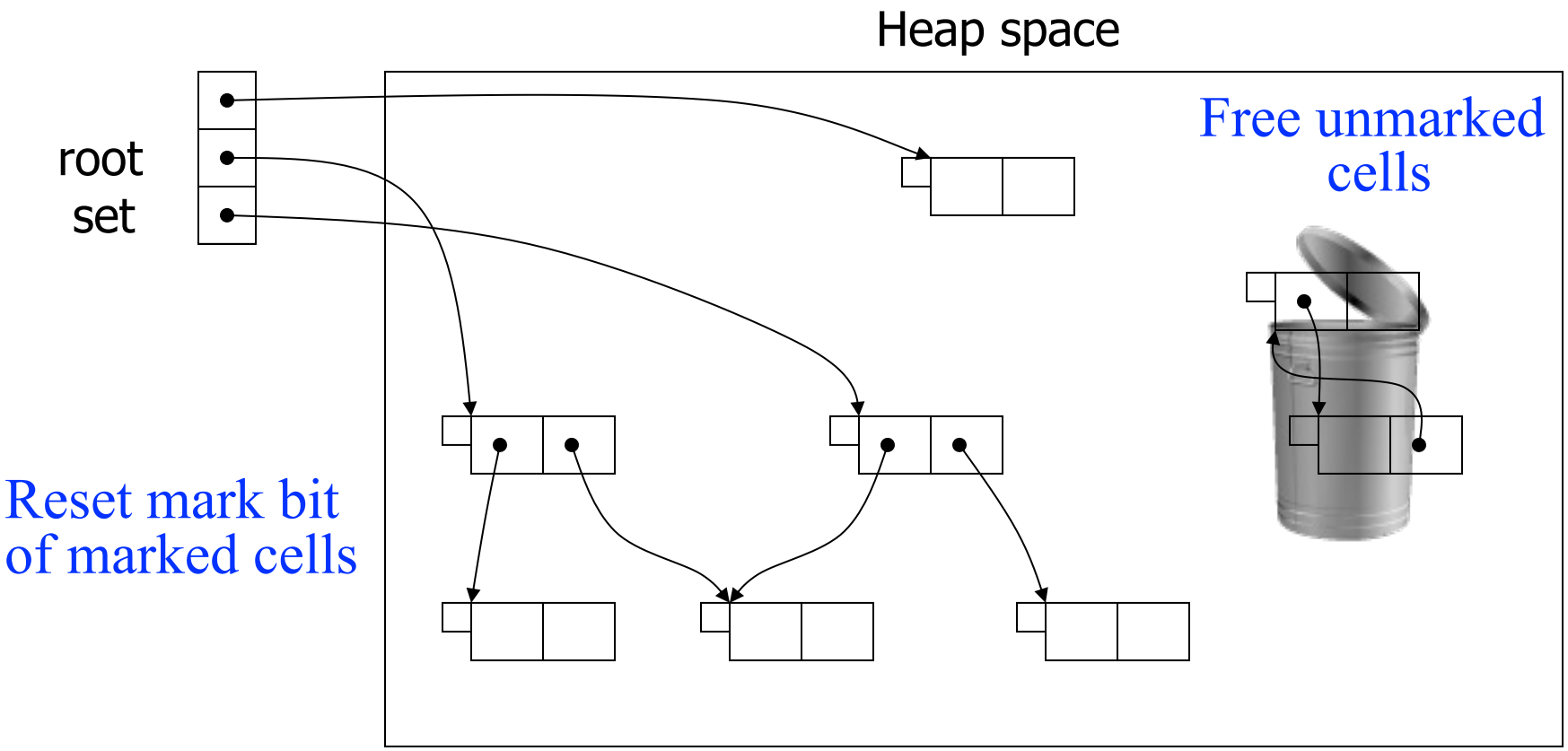
Example: Mark-and-Sweep



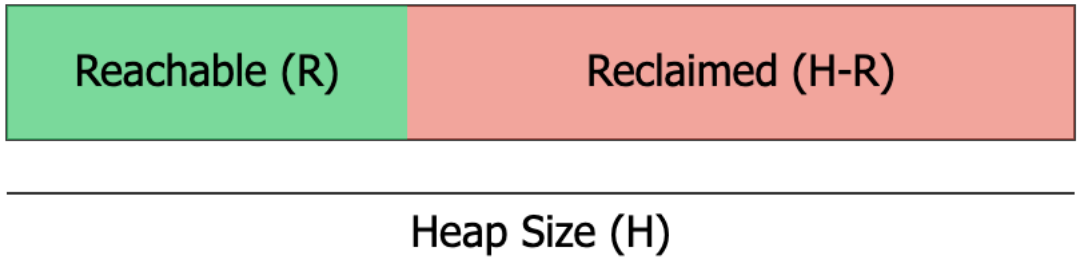
Example: Mark-and-Sweep



Example: Mark-and-Sweep



Cost of Mark-and-Sweep



Variables:

- **R** = words of reachable data
- **H** = total heap size (in words)
- **c_1** = marking cost per word
- **c_2** = sweeping cost per word

Cost Calculation:

- Total time: $c_1R + c_2H$
- Amortized cost per word allocated

  $(c_1R + c_2H) / (H - R)$

- If R is close to H , collection becomes very expensive
- If H is much larger than R, cost approaches c_2
- If R/H > 0.5 after collection, heap should be enlarged

1. Mark-and-Sweep

- Mark-and-Sweep
- Explicit Stack
- Pointer Reversal
- Memory Fragmentation

Implementation Issue of Mark-and-Sweep

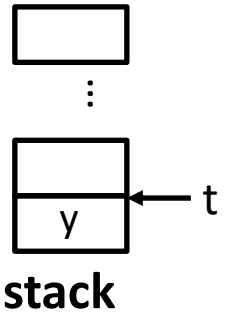
- Recursive DFS uses the **call stack**
- Worst case: longest path in reachable graph = H. Stack of **activation records** could be larger than the heap!
- **Irony:** The garbage collector, meant to manage memory, could itself run out of memory.😞
- Solutions:
  1. Explicit stack (save space, still O(H) worst case)
  2. Pointer reversal (O(1) extra space)

Solution 1: Using Explicit Stack

- Benefit: H words instead of H activation records !

```
function DFS(x)
    if x is a pointer and record x is not marked
        mark x
        t ← 1
        stack[t] ← x                    //把深搜的起点加入
        while t > 0
            x ← stack[t];  t ← t – 1    //取出栈顶元素
            for each field f_i of record x
                if x.f_i is a pointer and record x.fi is not marked
                    //指向了一个没有标记过的record
                    mark x.fi
                    t ← t + 1; stack[t] ← x.fi    // 加入栈中
```

- t: the top of the stack
- stack: a worklist



However, it is still unacceptable to require auxiliary stack memory as large as the heap being collected!

1. Mark-and-Sweep

- Mark-and-Sweep
- Explicit Stack
- Pointer Reversal
- Memory Fragmentation

Solution 2: Pointer Reversal

- **Problem**: Depth-first search needs a stack
  - Stack depth could be as big as the graph
  - Using explicit stack: still might need auxiliary memory as large as the heap

- **Solution:** pointer reversal
  - Known as the Deutsch-Schorr-Waite (DSW) algo.
  - Don't use an explicit stack for DFS
  - Reuse the graph components to assist backtracking

Developed independently by Schorr and Waite (1967) and by Deutsch (1973)

Pointer Reversal: The Idea

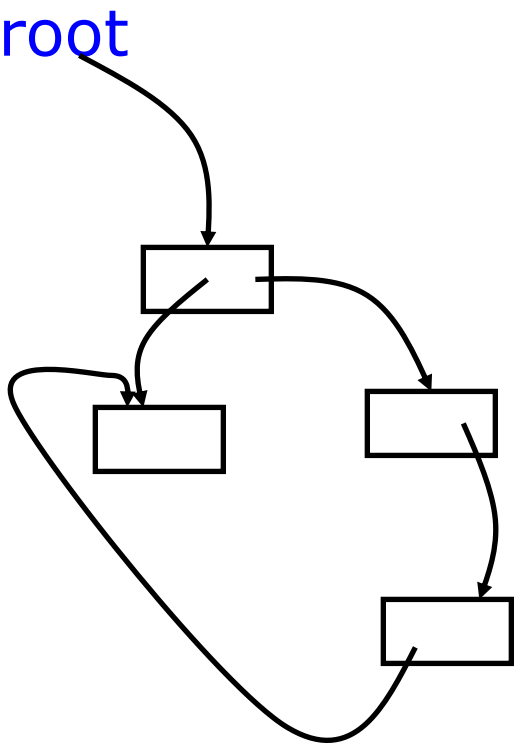
- **Goal:** Perform DFS with only **O(1)** extra space

- **Insight:** The graph we are traversing *is* the heap. We can temporarily **reuse its pointer fields** as our "stack"!

- **High-level idea**
  1. When descending to a child, **reverse the pointer** to point back to the parent.
  2. When all children processed, **follow the back-link** to return to the parent and **restore the original pointer**.

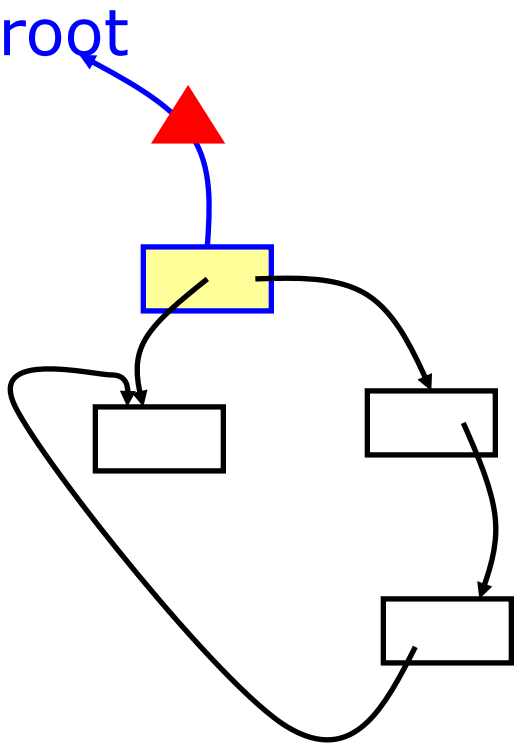
Pointer Reversal: The High-Level Idea

- Mark the record
- Change a pointer in the record to point back to the DFS parent record
- When going no deeper, return, following the back links, restoring the links



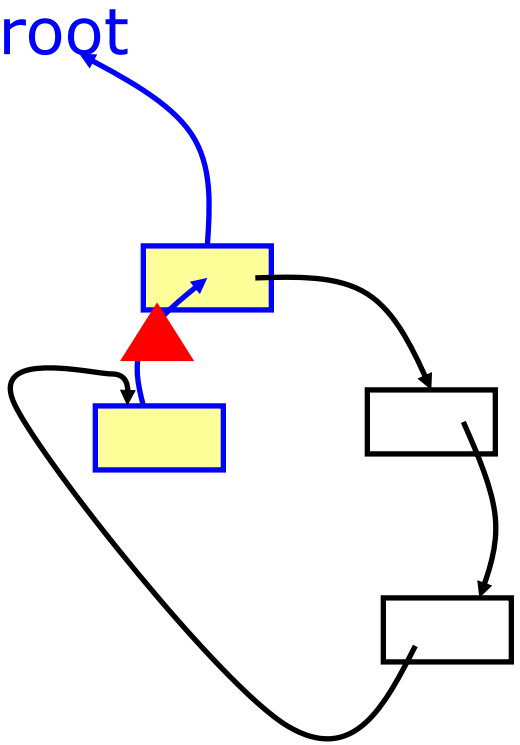
Example: Pointer Reversal

- Mark the record
- Change a pointer in the record to point back to the DFS parent record
- When going no deeper, return, following the back links, restoring the links



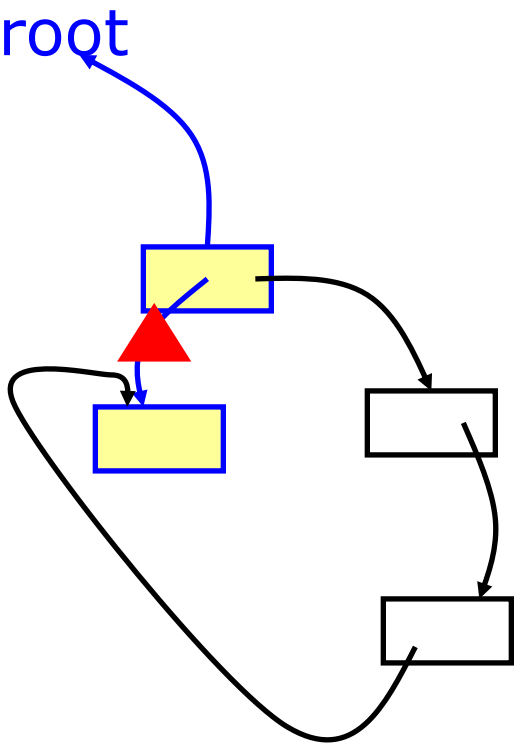
Example: Pointer Reversal

- Mark the record
- Change a pointer in the record to point back to the DFS parent record
- When going no deeper, return, following the back links, restoring the links



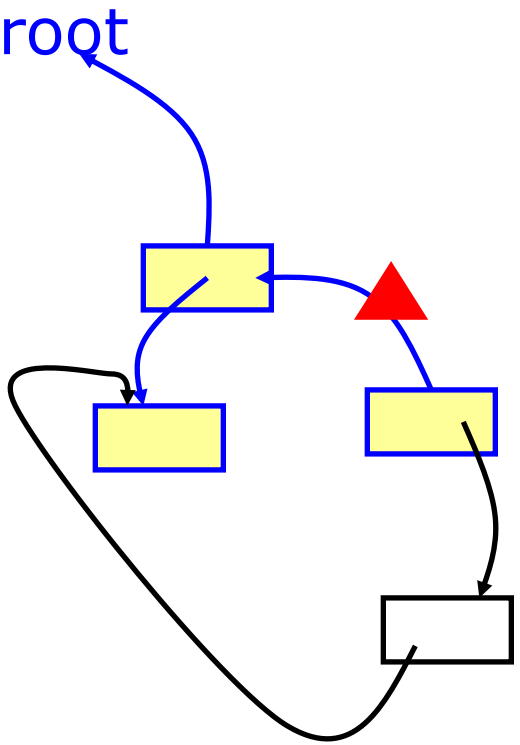
Example: Pointer Reversal

- Mark the record
- Change a pointer in the record to point back to the DFS parent record
- When going no deeper, return, following the back links, restoring the links



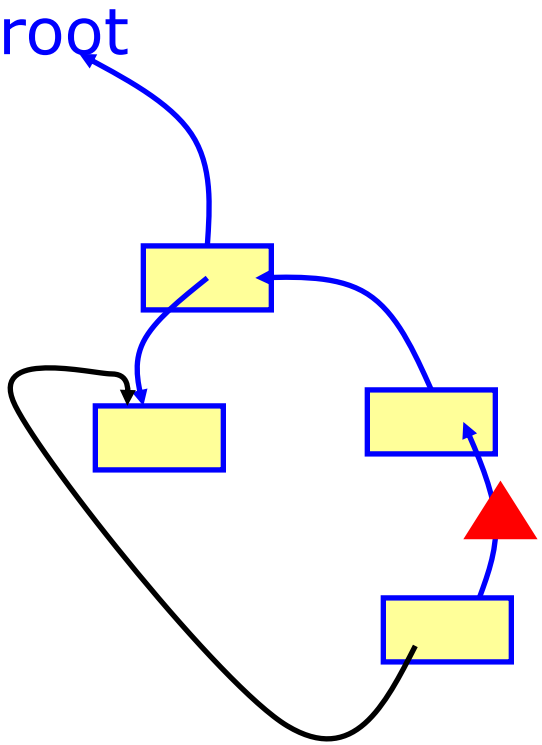
Example: Pointer Reversal

- Mark the record
- Change a pointer in the record to point back to the DFS parent record
- When going no deeper, return, following the back links, restoring the links.



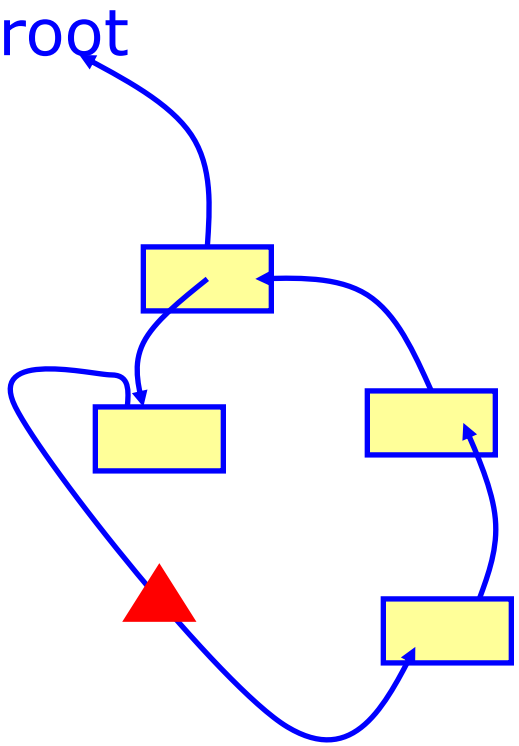
Example: Pointer Reversal

- Mark the record
- Change a pointer in the record to point back to the DFS parent record
- When going no deeper, return, following the back links, restoring the links.



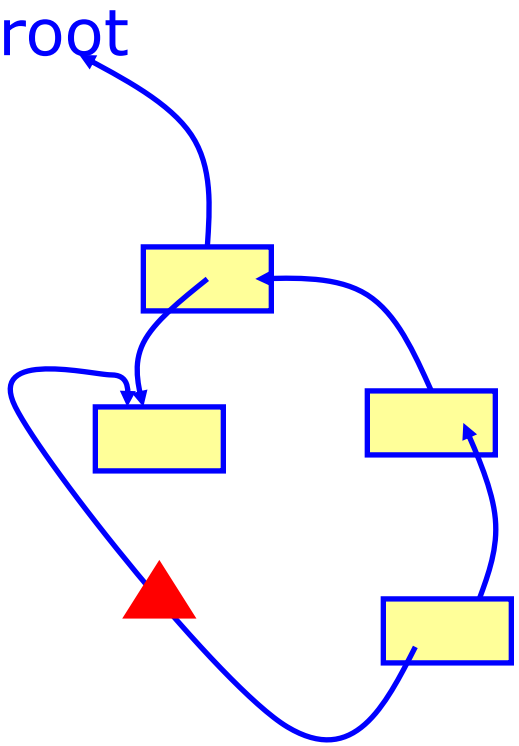
Example: Pointer Reversal

- Mark the record
- Change a pointer in the record to point back to the DFS parent record
- When going no deeper, return, following the back links, restoring the links.



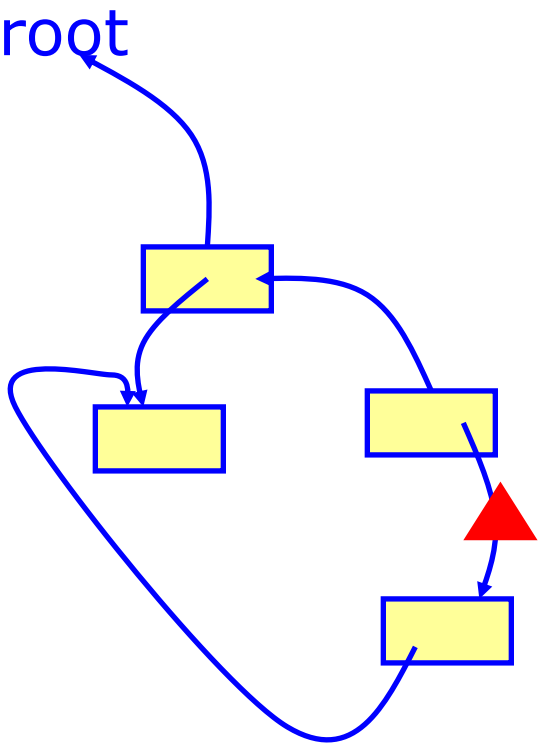
Example: Pointer Reversal

- Mark the record
- Change a pointer in the record to point back to the DFS parent record
- When going no deeper, return, following the back links, restoring the links



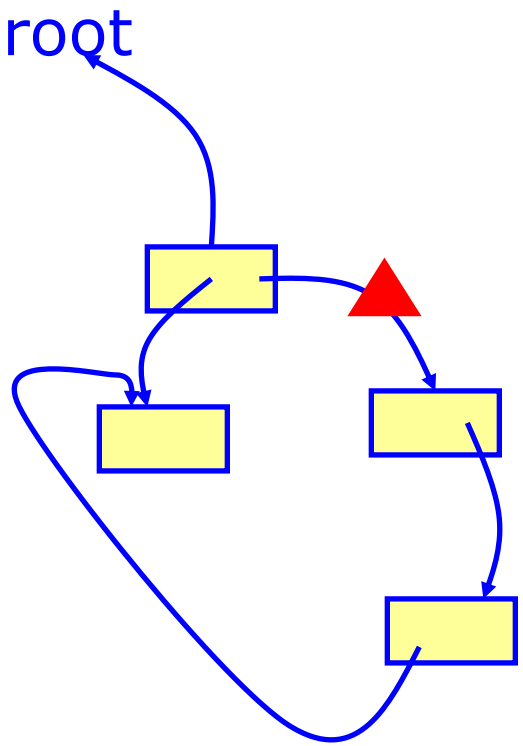
Example: Pointer Reversal

- Mark the record
- Change a pointer in the record to point back to the DFS parent record
- When going no deeper, return, following the back links, restoring the links



Example: Pointer Reversal

- Mark the record
- Change a pointer in the record to point back to the DFS parent record
- When going no deeper, return, following the back links, restoring the links



Pointer Reversal: The Algorithm

- After pushing *x.fi*, we *never again look the original x.fi*. again until we pop

- Eliminates the need for an external stack
- Uses **O(1) extra space** (just a few variables)
  - Each record needs a small done field (counts processed fields)
- On pop, the original $x.f_i$ value is restored

Mark phase:

```
for each root v
    DFS(v)

function DFS(x)
   if x is a pointer into the heap
    if record x is not marked
        mark x
        for each field fi of record x
            DFS(x.fi)
```

Use $x.f_i$ itself to store the "stack link" (pointer back to parent)!

Pointer Reversal Setup

```
function DFS(x)
 if x is a pointer and record x is not marked
  (* initialization *)

  while true
        ...
         if i < # of fields in record x

        (* process ith field *)



        else
           (* back-track to previous record *)
```

Call dfs passing each root as next

Object being processed

Back-track during the DFS
Decide termination here

Pointer Reversal Setup: Key Variables

Variable	Role
x	Current node being processed
t	Parent node (for backtracking); follows the reversed-pointer chain
done[x]	Number of fields already processed in node x
y	Temporary: holds child pointer or the node we just left

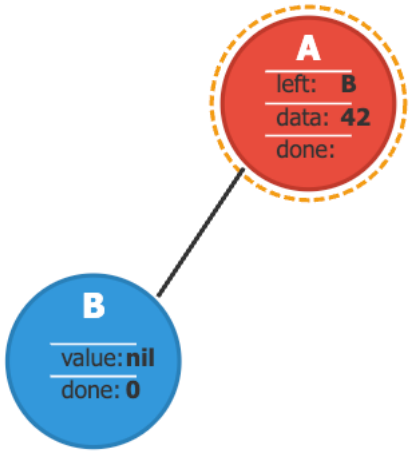
Total extra space: **O(1)** — just these few variables!
(Plus one done counter per heap record, stored in-place.)

Pointer Reversal: Full Algorithm

```
function DFS(x)
  if x is a pointer and record x is not marked
    t ← nil                                  // Initialize the "parent pointer"
    mark x; done[x] ← 0                      // Mark x and initialize its progress counter
    while true
          i ← done[x]                        // Get the current progress in this node
          if i < # of fields in record x     // If there are more fields to process
            y ← x. fi                        // Get the next field
            if y is pointer and record y not marked
              x. fi ← t; t ← x; x ← y        // Reverse pointer, move to child
              mark x; done[x] ← 0            // Mark child and initialize its counter
            else
              done[x] ← i + 1                // Move to next field in current node
          else
            y ← x; x ← t                     // Backtrack to parent
            if x = nil then return           // If we're back at the root, we're done
            i ← done[x]                      // Get parent's progress
            t ← x. fi; x. fi ← y             // Restore original pointer
            done[x] ← i + 1                  // Move to next field in parent
```

Example: Initialization

```
function DFS(x)
  if x is a pointer and record x is not marked
    t ← nil
    mark x; done[x] ← 0
    while true
          i ← done[x]
          if i < # of fields in record x
             y ← x. fi
             if y is pointer and record y not marked
                x. fi ← t; t ← x; x ← y
                mark x; done[x] ← 0
             else
                done[x] ← i + 1
          else
             y ← x; x ← t
             if x = nil then return
             i ← done[x]
             t ← x. fi; x. fi ← y
             done[x] ← i + 1
```



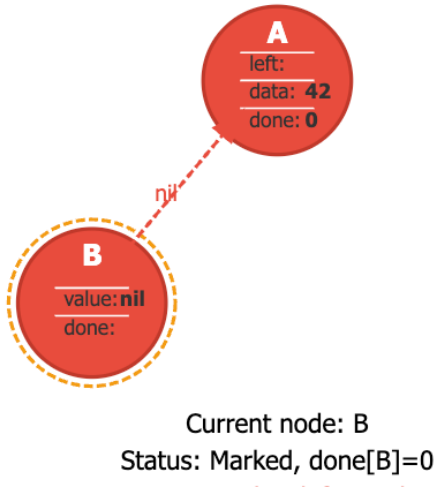
- Mark node A (shown in red)
- Set done[A] = 0 (no fields processed yet)
- Set t = nil (no parent)

Variable	Value
x	A
t	nil
done[A]	0

Next, we'll process A's left field.

Example: Process A's Left Field (Pointer to B)

```
function DFS(x)
  if x is a pointer and record x is not marked
    t ← nil
    mark x; done[x] ← 0
    while true
          i ← done[x]
          if i < # of fields in record x
             y ← x. fi
             if y is pointer and record y not marked
                x. fi ← t; t ← x; x ← y
                mark x; done[x] ← 0
             else
                done[x] ← i + 1
          else
             y ← x; x ← t
             if x = nil then return
             i ← done[x]
             t ← x. fi; x. fi ← y
             done[x] ← i + 1
```

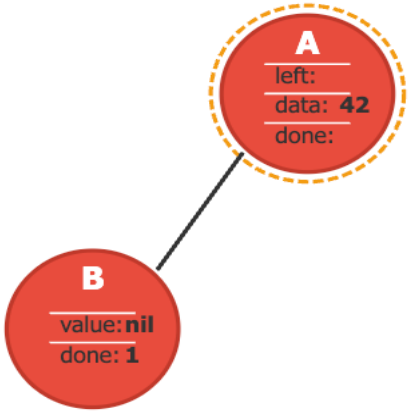


We have replaced A's left pointer with nil and stored A in t

Variable	Value
x	B (was A)
t	A (was nil)
i	0
y	B (value of A.left)
done[A]	0
done[B]	0

Example: Process B and Backtrack to A

```
function DFS(x)
  if x is a pointer and record x is not marked
    t ← nil
    mark x; done[x] ← 0
    while true
          i ← done[x]
          if i < # of fields in record x
             y ← x. fi
             if y is pointer and record y not marked
                x. fi ← t; t ← x; x ← y
                mark x; done[x] ← 0
             else
                done[x] ← i + 1
          else
             y ← x; x ← t
             if x = nil then return
             i ← done[x]
             t ← x. fi; x. fi ← y
             done[x] ← i + 1
```



Current node: A
Status: done[A]=1

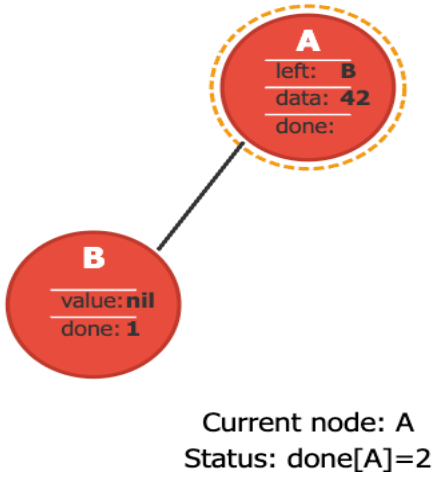
1.Process B's value field:

2.Backtrack to A:

Variable	Value
x	B
t	A
i	0
y	nil (B.value)
done[A]	0
done[B]	1 (was 0)

Example: Process A's Data Field (Non-Pointer)

```
function DFS(x)
  if x is a pointer and record x is not marked
    t ← nil
    mark x; done[x] ←  0
    while true
          i ← done[x]
          if i < # of fields in record x
             y ← x. fi
             if y is pointer and record y not marked
                x. fi ← t; t ← x; x ← y
                mark x; done[x] ← 0
             else
                done[x] ← i + 1
          else
             y ← x; x ← t
             if x = nil then return
             i ← done[x]
             t ← x. fi; x. fi ← y
             done[x] ← i + 1
```

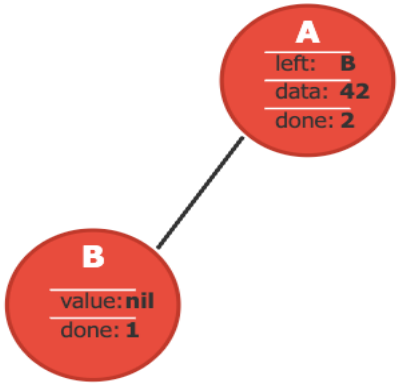


Simply increment the done counter and continue.

Variable	Value
x	A
t	nil
i	1
y	42 (A.data)
done[A]	2 (was 1)
done[B]	1

Example: Complete Traversal

```
function DFS(x)
  if x is a pointer and record x is not marked
    t ← nil
    mark x; done[x] ← 0
    while true
          i ← done[x]
          if i < # of fields in record x
             y ← x. fi
             if y is pointer and record y not marked
                x. fi ← t; t ← x; x ← y
                mark x; done[x] ← 0
             else
                done[x] ← i + 1
          else
             y ← x; x ← t
             if x = nil then return
             i ← done[x]
             t ← x. fi; x. fi ← y
             done[x] ← i + 1
```



Traversal Complete!
All nodes marked: A, B

Variable	Value
x	nil (was A)
t	-
i	-
y	A
done[A]	2
done[B]	1

Summary: Pointer Reversal

Key Algorithm Steps

1.Forward traversal:

- Replace child pointers with "breadcrumbs" back to parent
- Save the parent in t
- Move to child

2. Non-pointer fields:

- Simply increment done counter

3. Backtracking:

- Use t to return to parent
- Restore original pointers
- Move to next field in parent

Key Variables

- **x**: Current node being processed
- **t**: Parent node to return to (backtracking)
- **done**: Number of fields processed in node
- **y**: Temporary storage for child node or field value

1. Mark-and-Sweep

- Mark-and-Sweep
- Explicit Stack
- Pointer Reversal
- Memory Fragmentation

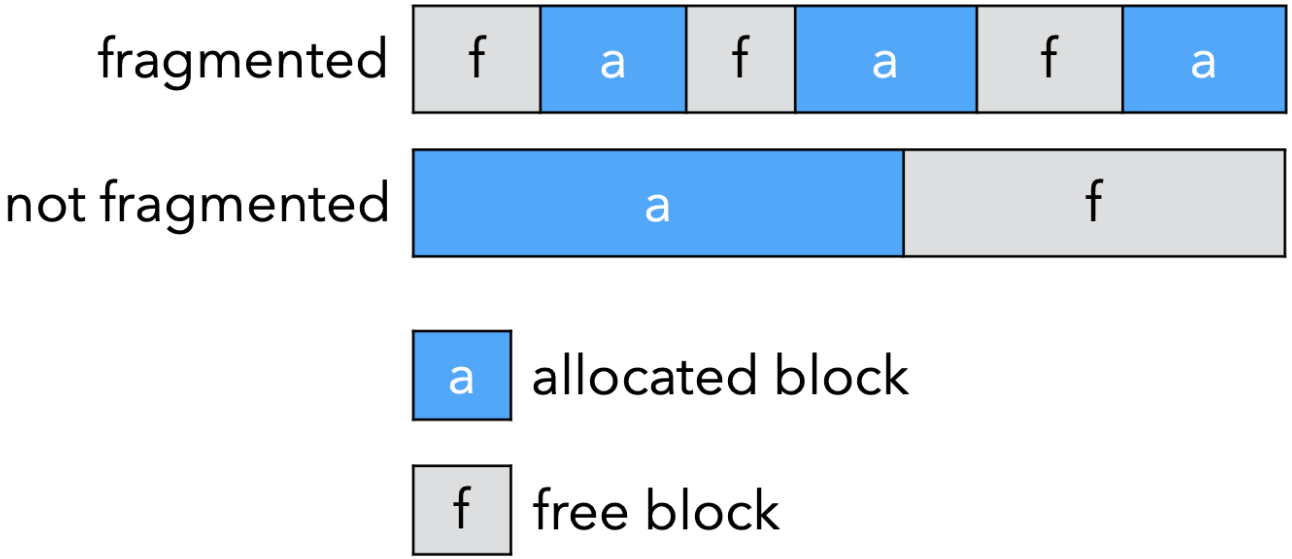
The Fragmentation Problem

- **External fragmentation**
  - Program wants to allocate a record of size n, but there are many free records smaller than n

- **Internal fragmentation**
  - Program uses a too-large record without splitting it, so that the unused memory is inside the record instead of outside

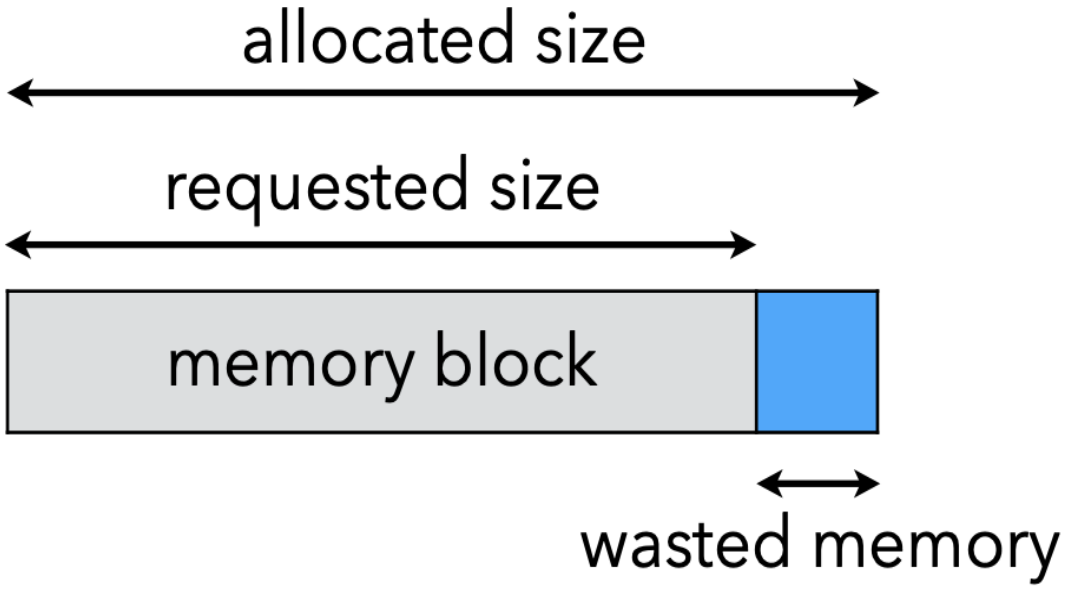
Example: External Fragmentation

- The two heaps below have the same amount of free memory, but the first suffers from **external fragmentation** while the second does not.
- Thus, some requests can be fulfilled by the second but not by the first



Example: Internal Fragmentation

- The memory manager sometimes allocates more memory than requested, e.g. to satisfy alignment constraints.
- This results in small amounts of wasted memory scattered in the heap, called **internal fragmentation.**



Summary: Mark-and-Sweep

- **Pros**
  - High efficiency if little garbage exist.
  - Be able to collect cyclic references.
  - Objects/records are not moved during GC

- **Cons**
  - Low efficiency with large amount of garbage
  - Normal execution must be suspended
  - Leads to fragmentation in the heap (Cache misses, page thrashing, complex allocation, etc.)

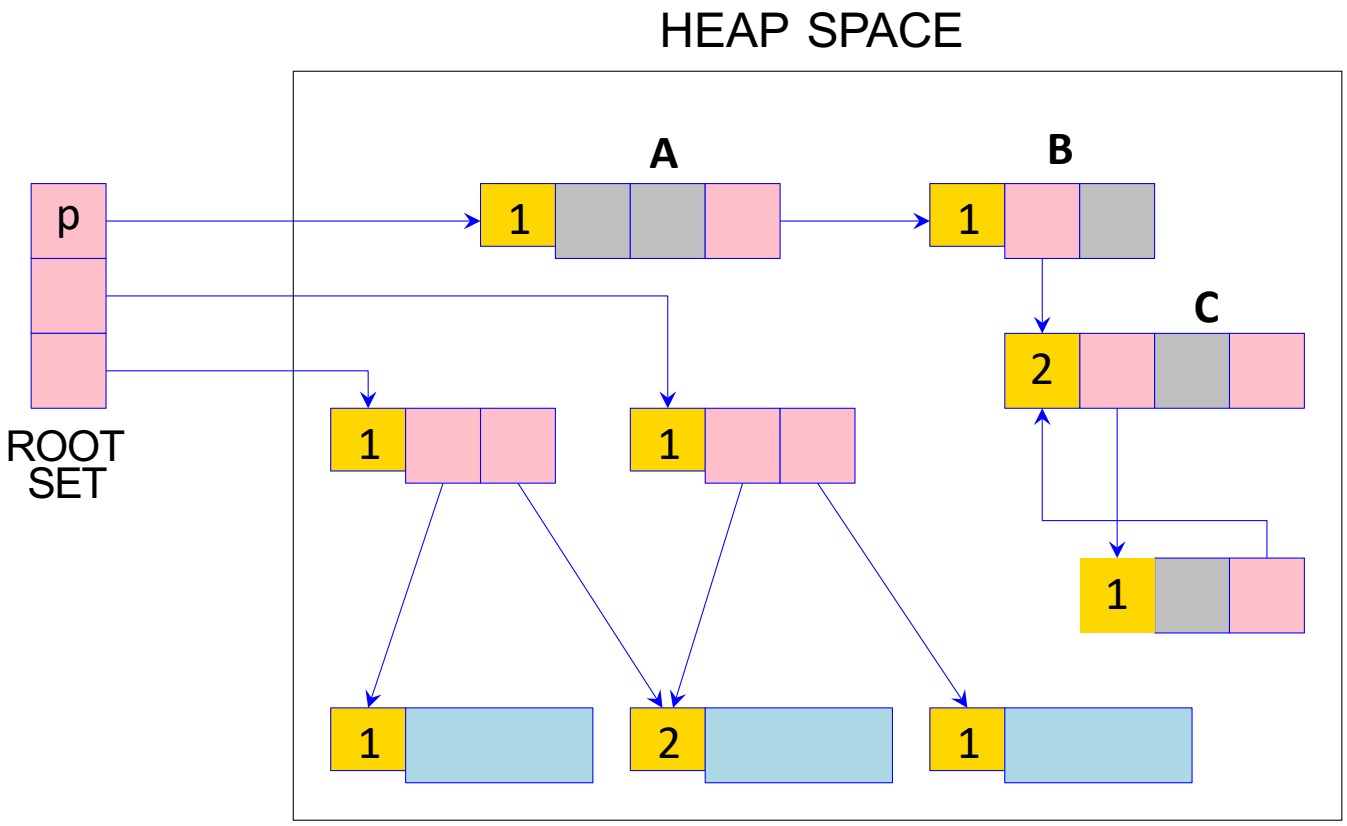
2. Reference Counting

Reference Counting

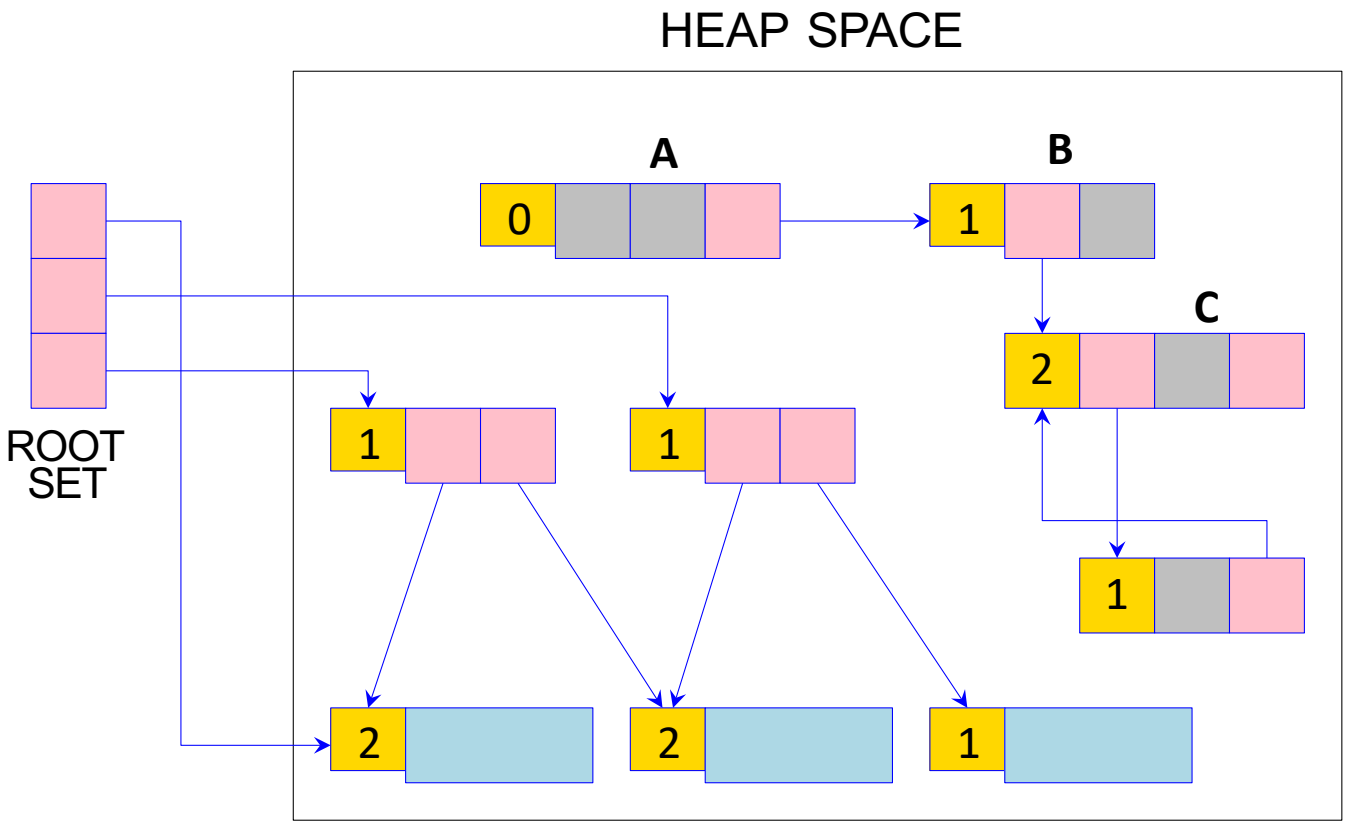
- Keep track of how many pointers point to each record (reference count)
- When count reaches zero, record is garbage and can be reclaimed
- More eager than mark-and-sweep: reclaims as soon as unreachable

reference_count(x) = 0 ➔ x is unreachable

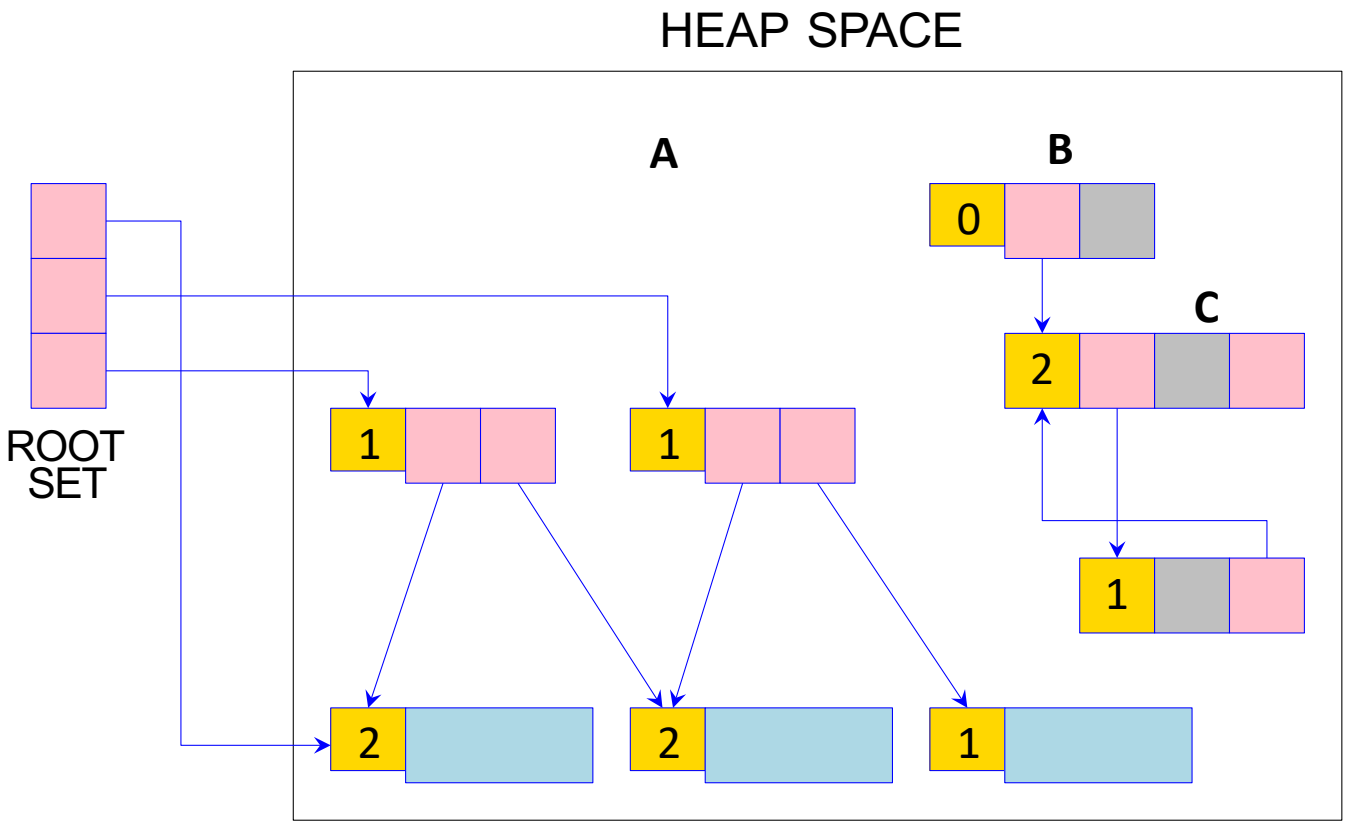
Example: Reference Counting



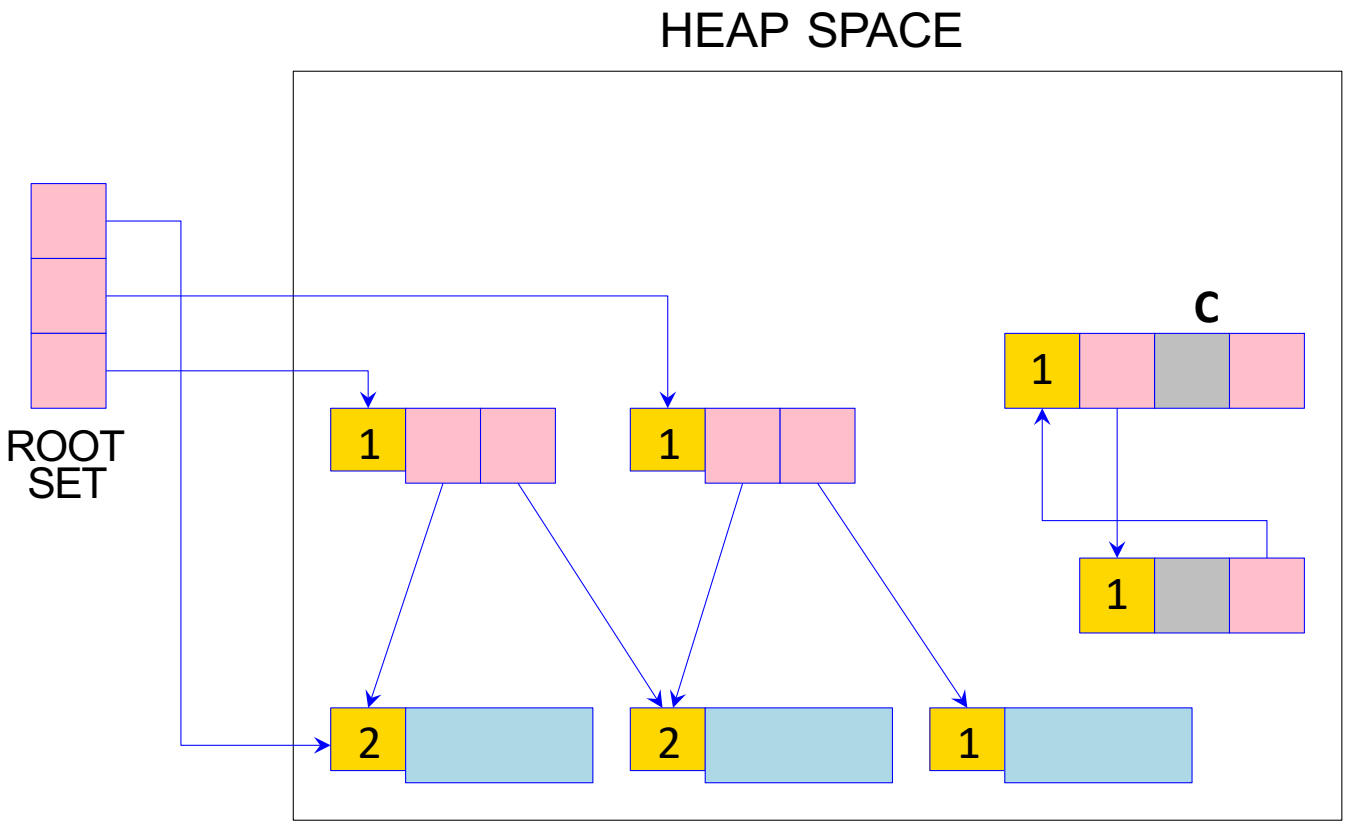
Example: Reference Counting



Example: Reference Counting



Example: Reference Counting



Reference Counting

- How to keep track? E.g., each assignment operation manipulates the reference counts.
- **Whenever p is stored into x.fi (i.e., x.fi = p)**
  1. The reference count of p is incremented, and
  2. The reference count of what x.fi previously pointed to is decremented.
- **If the reference count of some record r reaches zero**
  - r is put on the *freelist*, and
  - all the other records that r points to have their reference counts decremented.

Reference Counting: Strengths

- Incremental overhead: Cell management interleaved with program execution
  - So, no "stop-and-collection" effect
- Can coexist with manual memory management
- Can re-use freed cells immediately
  - If RC == 0, put back onto the free list

Reference Counting: Limitations

- Reference counting seems simple and attractive.

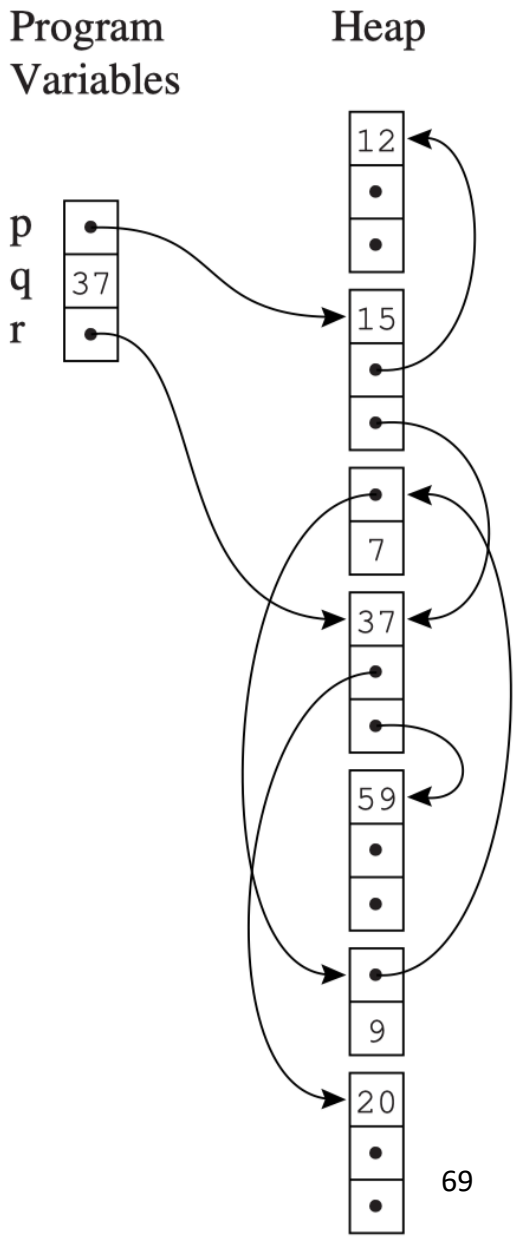
- But there are two major problems:

1. Cycles of garbage cannot be claimed
2. Incrementing the reference counts is very expensive

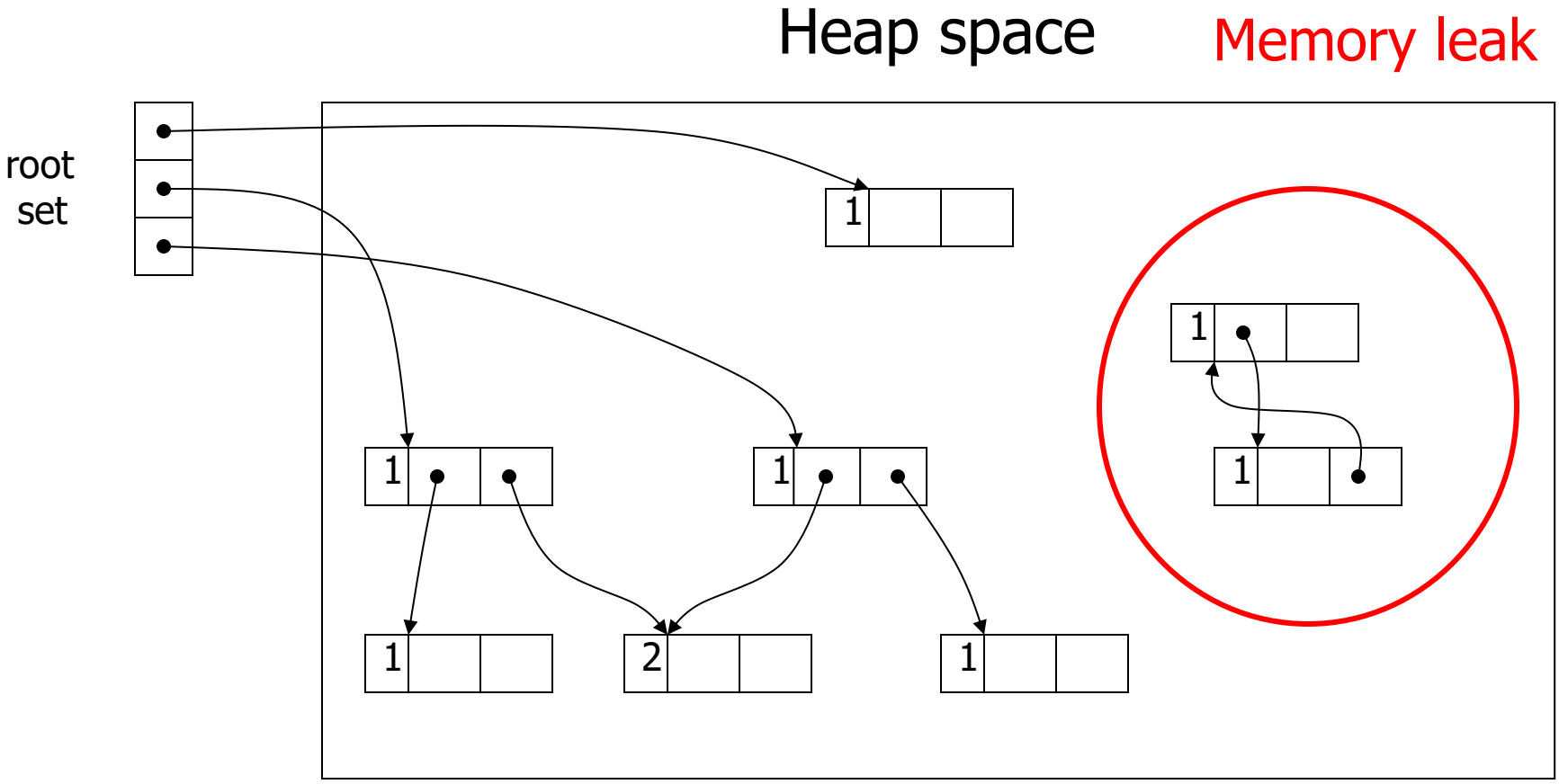
Problem 1: Reference Cycle

- A reference cycle is a set of objects that cyclically refer to one another.
  - Example: the record storing 7 and the record storing 9

- Reference count tracks the number of references, not number of reachable references.



Example: Reference Counting: Cycles



Problem 2: Performance Overhead

- Incrementing the reference counts is very expensive.
- In place of the single machine instrution *x. fi* ← *p*, the program must execute:

```
z ← x.fi                          // Save original value
c ← z.count                       // Get current count
c ← c - 1                         // Decrement ref. count of x.fi
z.count ← c                       // Store back
if c = 0 then putOnFreelist(z)    // Free if zero
x.fi ← p                          // Perform the assignment
c ← p.count                       // Get current count
c ← c + 1                         // Increment reference count of p
p.count ← c                       // Store back
```

Summary: Reference Counting

- **Advantages**
  - Immediate collection: (i.e., reduce the time between the object becoming garbage and its reclaimation)
  - Incremental collection
  - Simple to implement

- **Disadvantages**
  - Can not collect unreachable cyclic data structures
  - Can be relatively inefficient

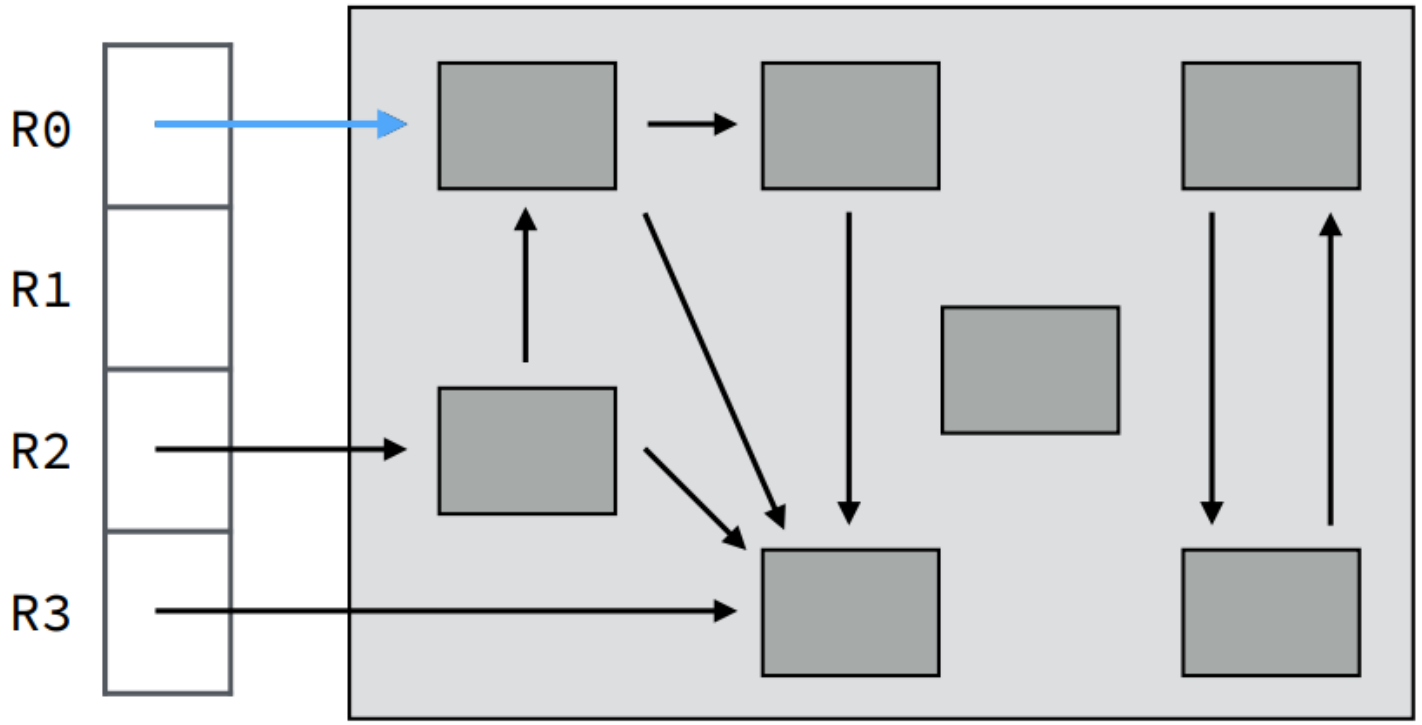
Improving Reference Counting (供了解)

- **Solutions for Cycles**
  - Hybrid GC: Use reference counting most of the time, but periodically run mark-sweep to collect cycles

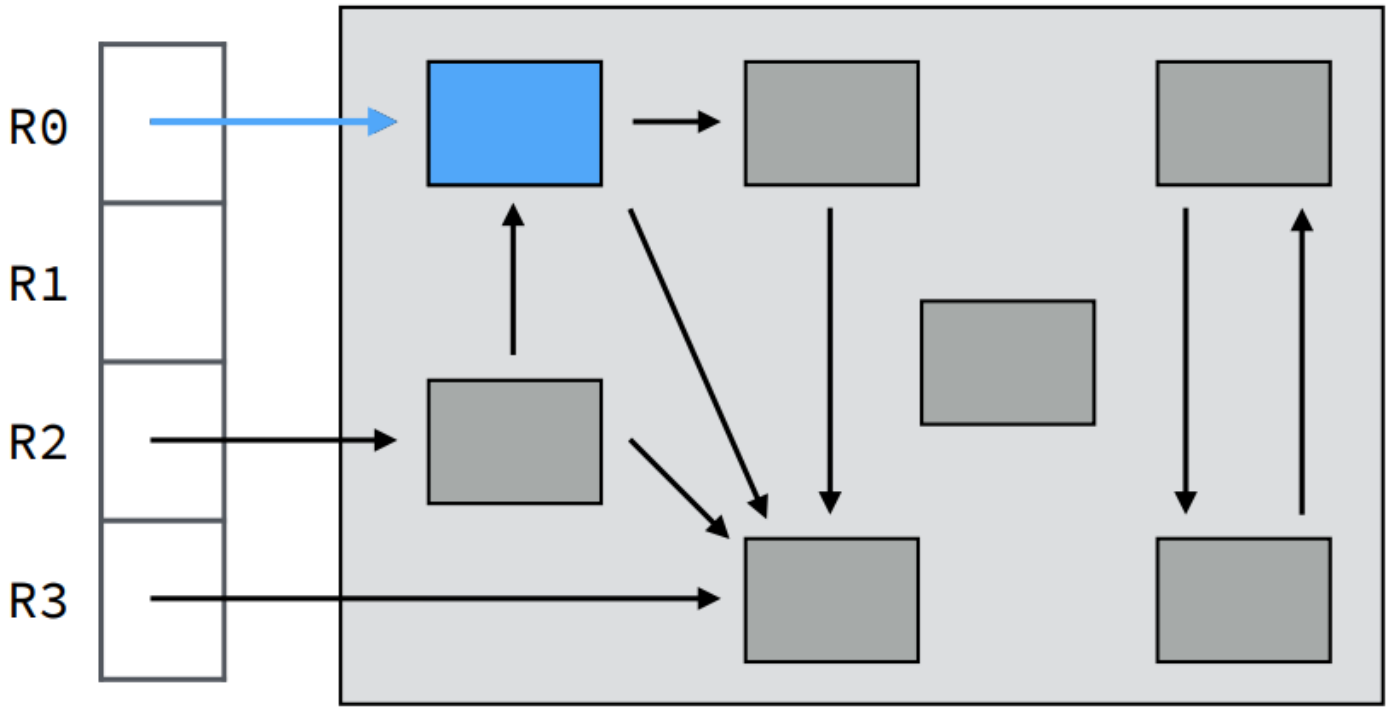
- **Optimization Performance**
  - **Static analysis** for eliminating redundant updates
  - **Local variable optimization:** Don't count references from stack variables
  - **Hardware support:** Some architectures provide atomic reference counting operations

Thank you all for your attention

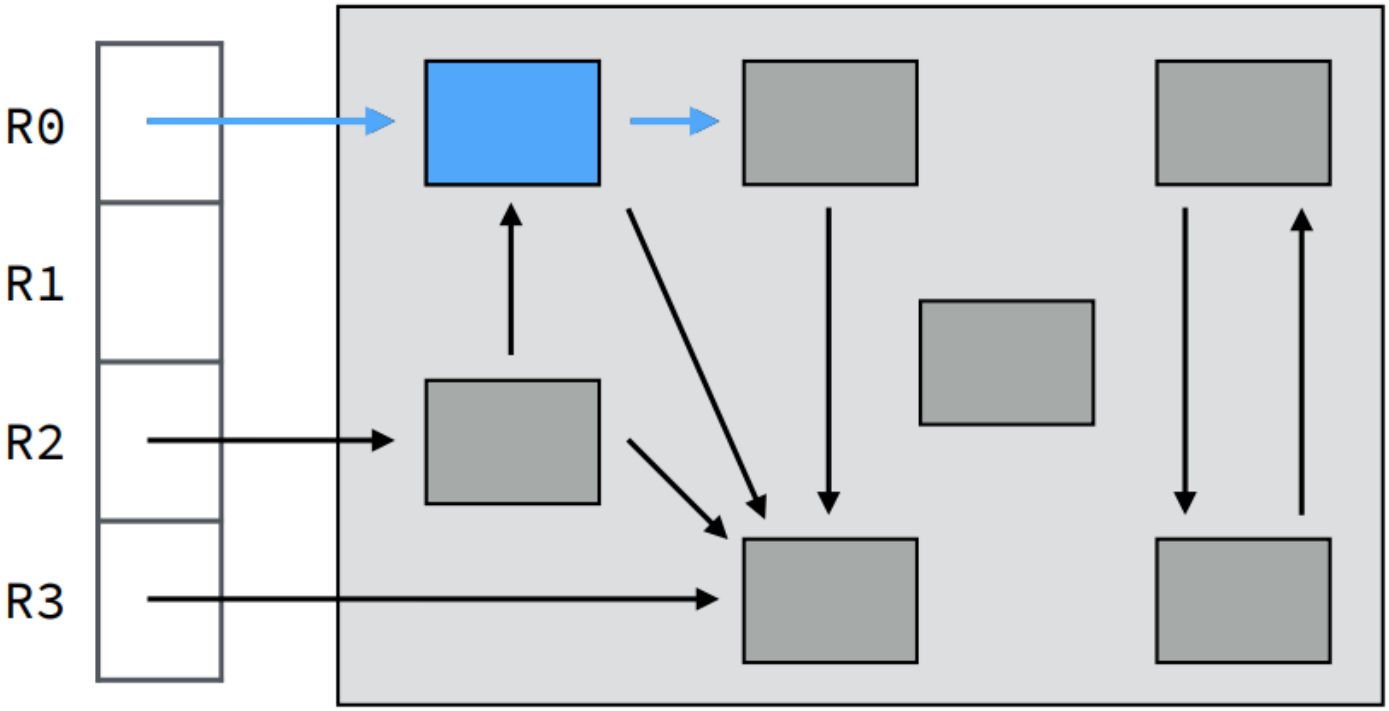
Example: Mark-and-Sweep



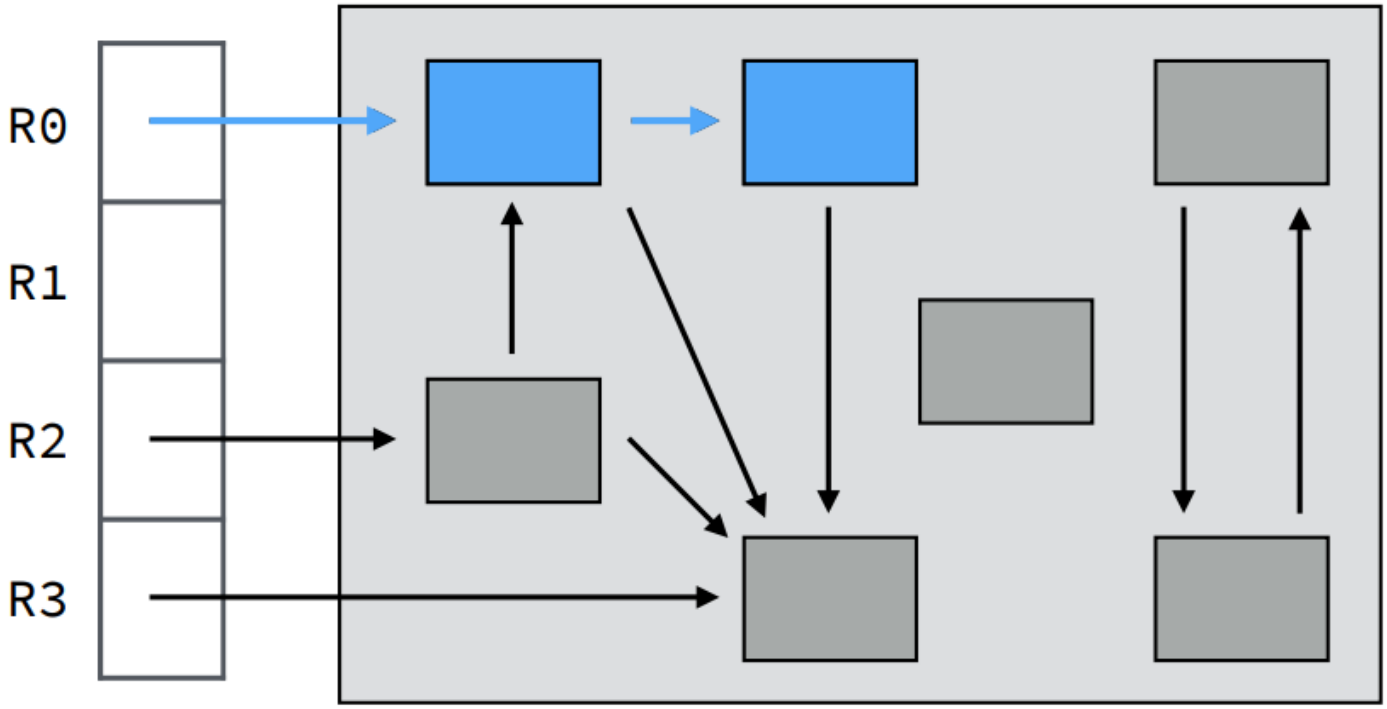
Example: Mark-and-Sweep



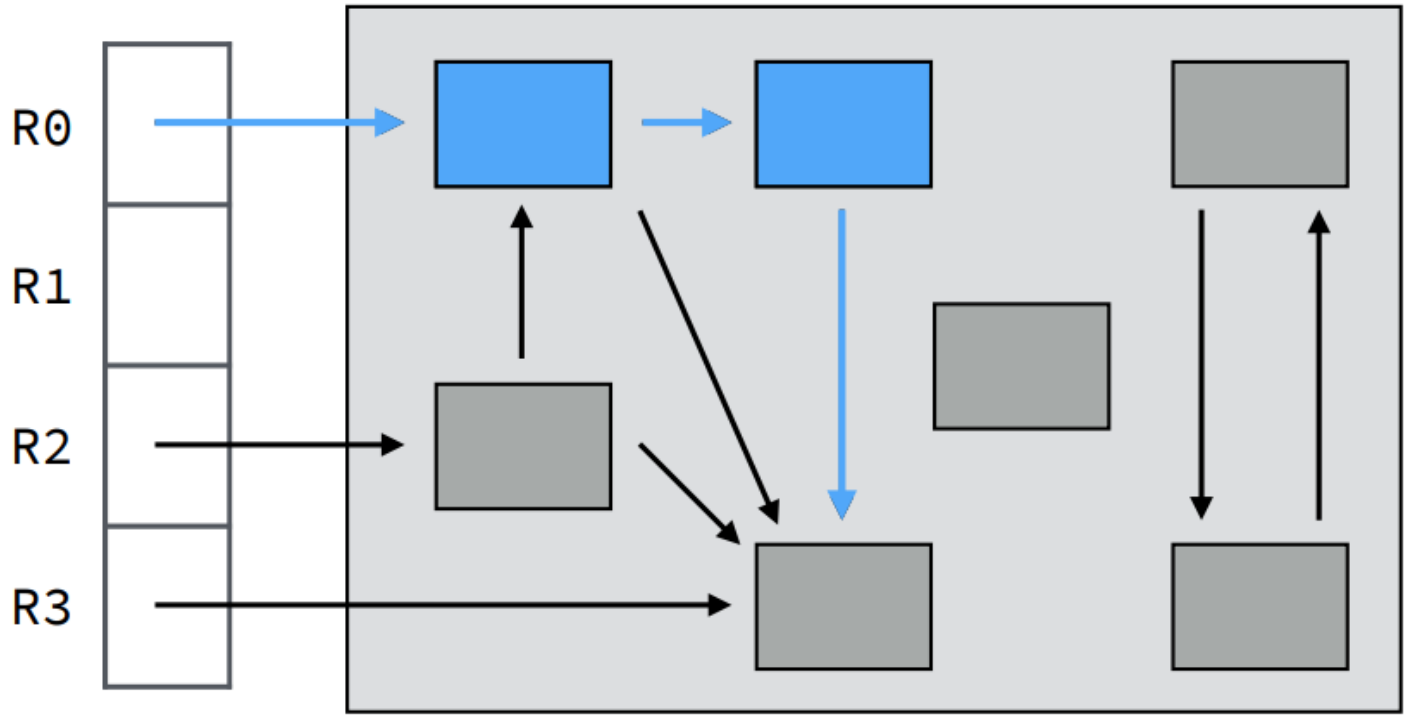
Example: Mark-and-Sweep



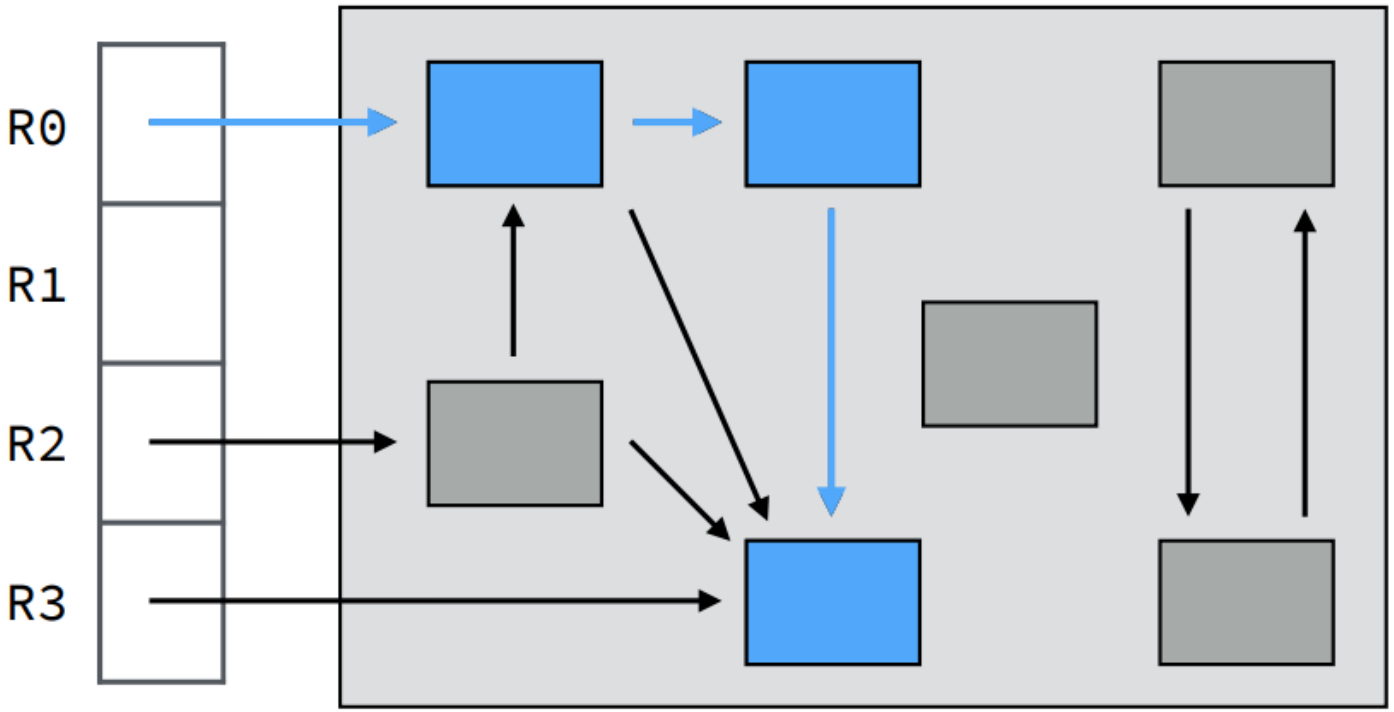
Example: Mark-and-Sweep



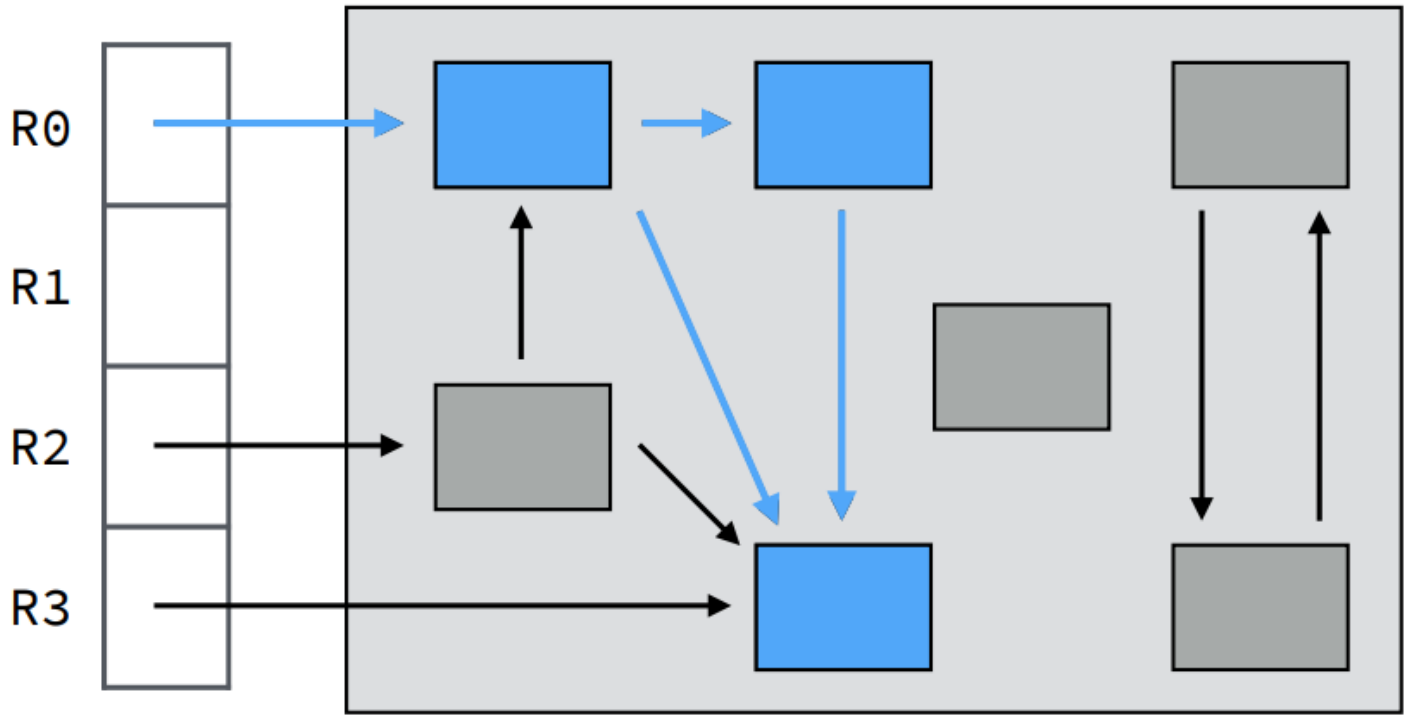
Example: Mark-and-Sweep



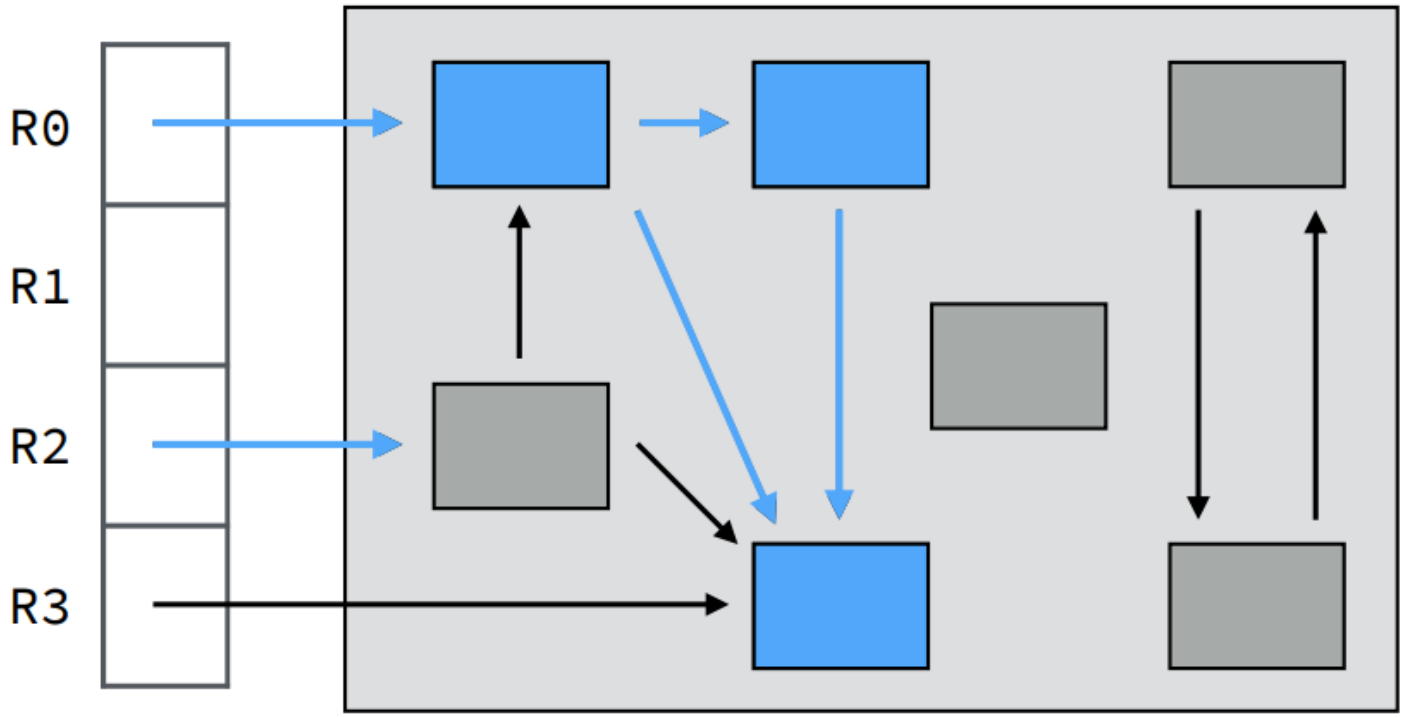
Example: Mark-and-Sweep



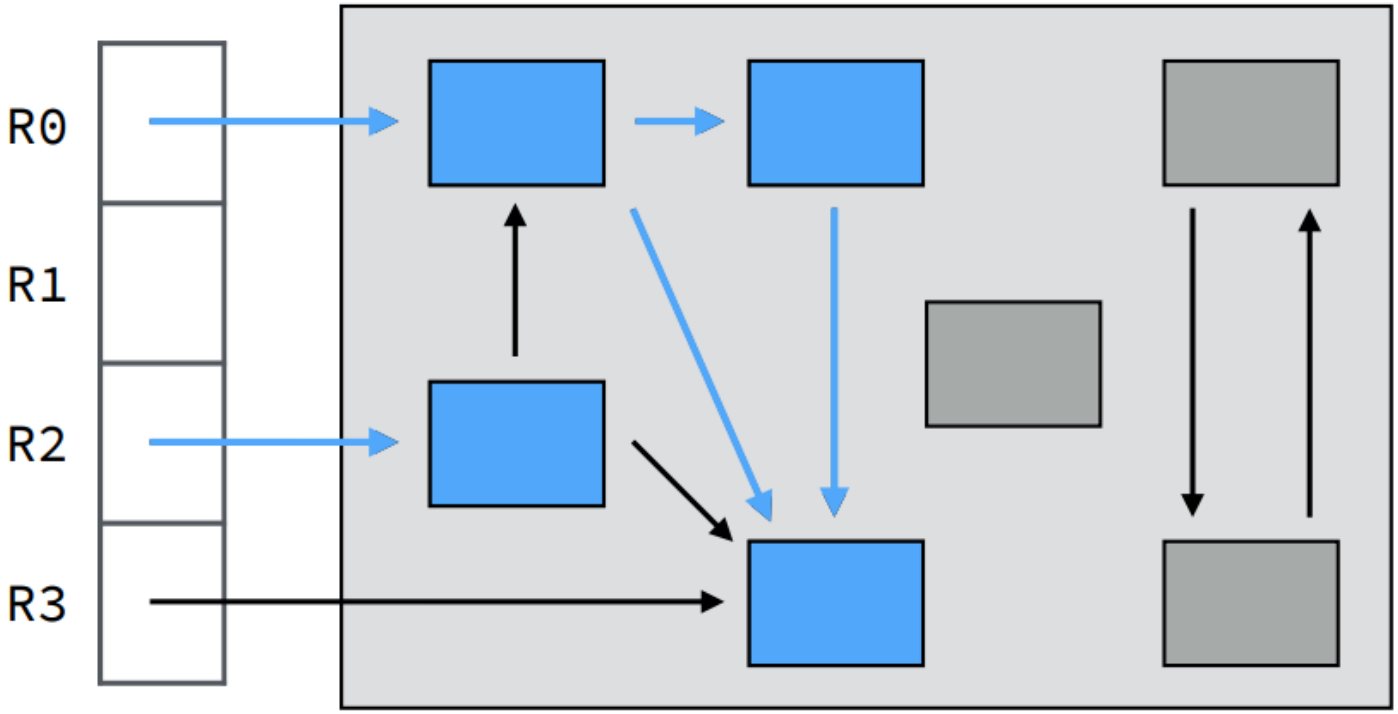
Example: Mark-and-Sweep



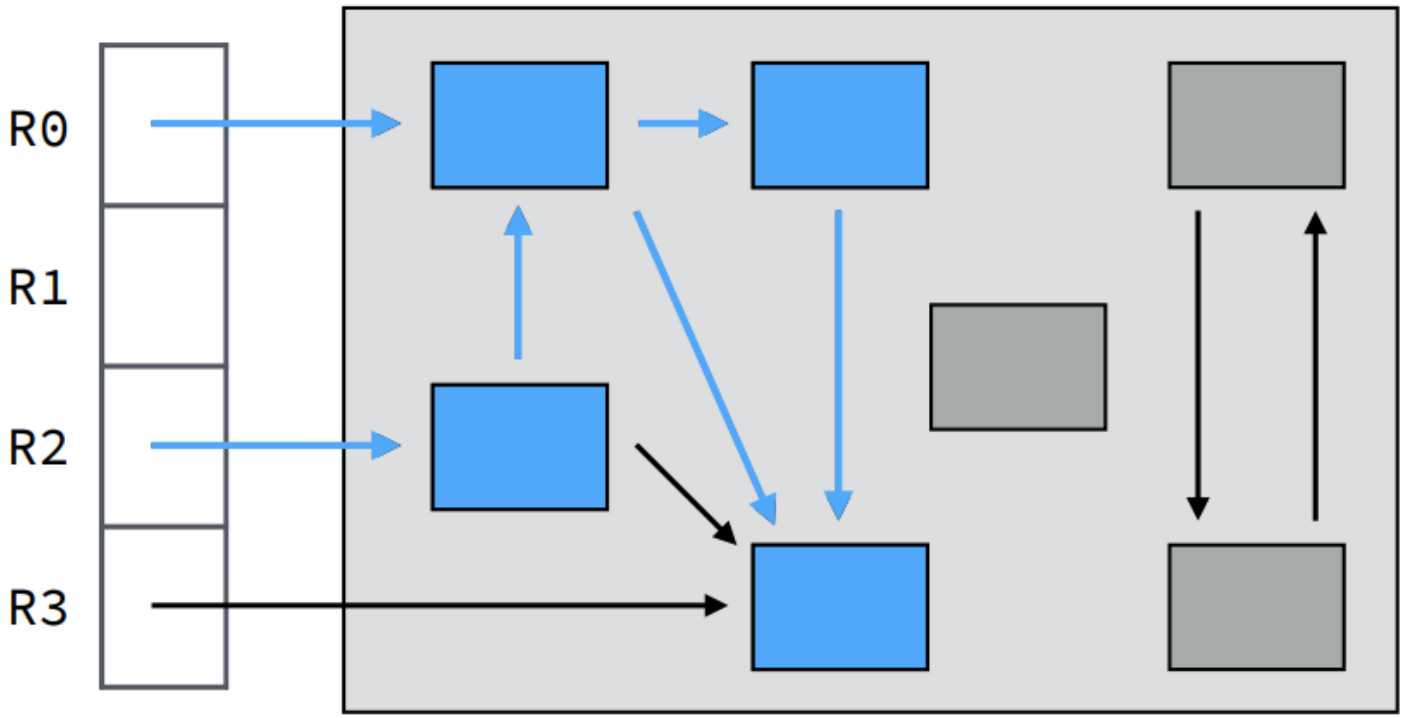
Example: Mark-and-Sweep



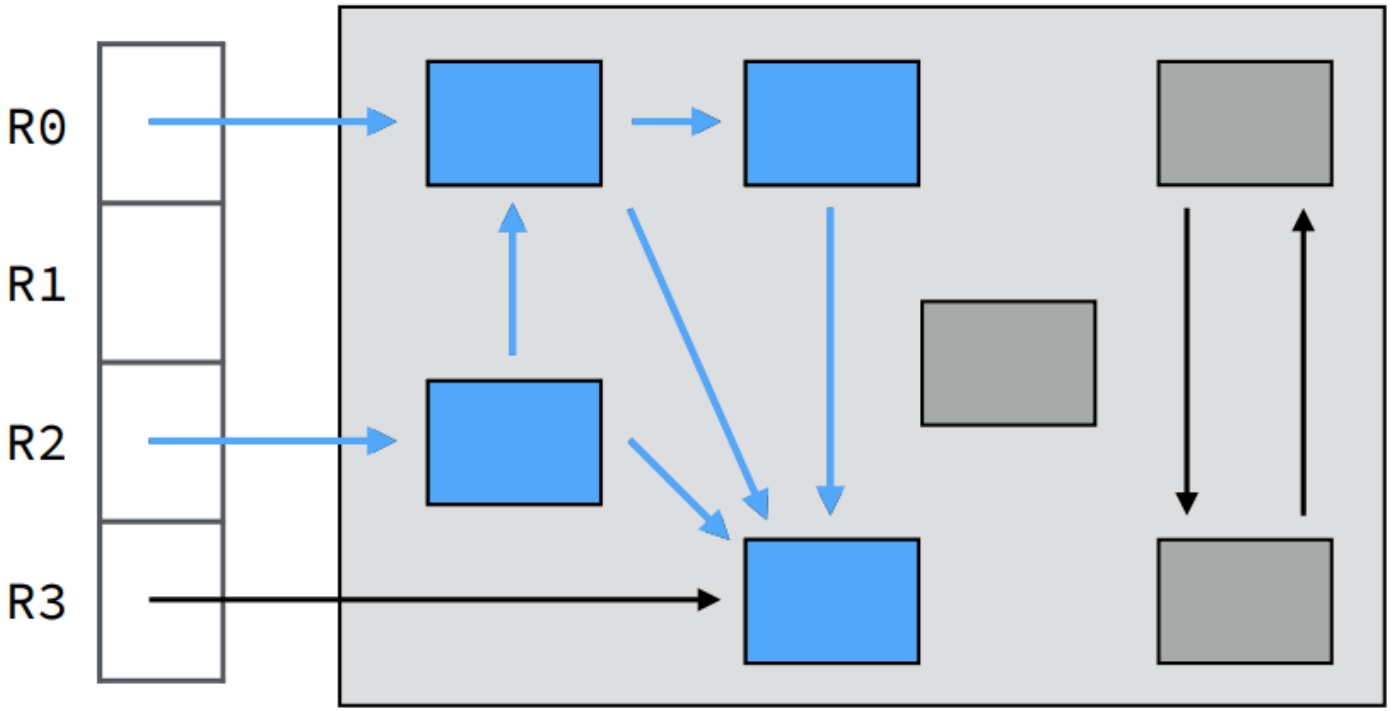
Example: Mark-and-Sweep



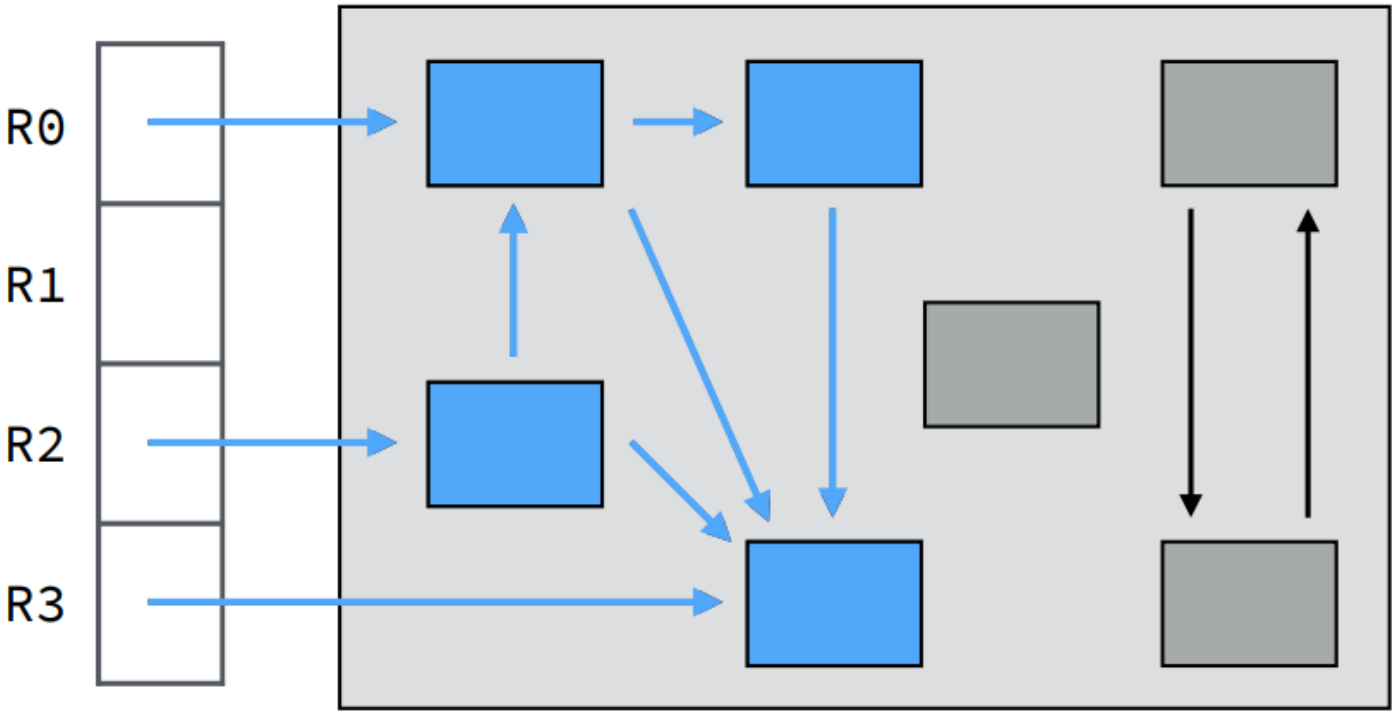
Example: Mark-and-Sweep



Example: Mark-and-Sweep



Example: Mark-and-Sweep



Example: Mark-and-Sweep